

FREE EBOOK

The Web API Production Checklist

33 things to get right before your
ASP.NET Core Web API goes to
production.



33 CHECKS • ONE PASS BEFORE YOU SHIP

Table of contents

Introduction

The checklist: all 33 items on one page

Chapter 1: Foundations

- 1.1 Put controllers behind services, not straight on top of your DbContext
- 1.2 Register every service with the lifetime it actually needs
- 1.3 Never expose your entities; return DTOs
- 1.4 Get your middleware order right, and know what each one does

Chapter 2: Contracts & Validation

- 2.1 Validate every incoming payload; never trust the client
- 2.2 Move validation out of your actions into a filter
- 2.3 Reach for FluentValidation when rules outgrow attributes
- 2.4 Return the right status code for the right situation
- 2.5 Support PATCH properly if you claim to support it
- 2.6 Make your verbs safe and idempotent the way HTTP says they are

Chapter 3: Error Handling

- 3.1 Handle exceptions globally; delete every try/catch in your controllers
- 3.2 Return RFC 7807 problem details, not ad-hoc error strings
- 3.3 Model expected failures as typed exceptions, not magic nulls

Chapter 4: Security

- 4.1 Authenticate with JWT bearer tokens and validate all four parameters
- 4.2 Keep the signing secret out of appsettings and out of source control
- 4.3 Bind configuration to typed options instead of stringly-typed lookups
- 4.4 Use ASP.NET Core Identity for users; do not hand-roll password storage
- 4.5 Authorize by role or policy on every non-public endpoint
- 4.6 Issue short-lived access tokens with refresh tokens behind them
- 4.7 Configure CORS deliberately, not AllowAnyOrigin

Chapter 5: Performance & Scale

- 5.1 Never return an unbounded collection; page everything
- 5.2 Make queries async and read-only by default
- 5.3 Filter, search, and sort on the server, not in the client
- 5.4 Validate anything you interpolate into a dynamic query
- 5.5 Cache responses that can tolerate being slightly stale
- 5.6 Rate limit before someone finds out you did not

Chapter 6: Observability & Documentation

- 6.1 Log through a structured logger, and never log secrets
- 6.2 Expose a health check endpoint your orchestrator can actually use
- 6.3 Ship real OpenAPI docs, with auth and response types wired in

Chapter 7: Ship It

- 7.1 Version your API from day one
- 7.2 Test the API through its real pipeline, not around it
- 7.3 Separate configuration per environment and keep production values out of the repo
- 7.4 Deploy from a published build and let the host supply the secrets

Where to go next

Introduction

You can write C#. You know what a controller is, you have used Entity Framework Core, and you have an API that works on your machine. What you have not done yet is put one in front of real users, real load, and real people who will call it in ways you did not plan for.

That gap is what this book is about. Almost nothing in it is hard. It is a list of the decisions that are cheap to make on day one and expensive to retrofit on day two hundred. Those are the ones that separate an API that works from an API that keeps working when it is not you making the requests.

None of these items are exotic. Most of them are things you will nod at. The reason they still bite people is that every one of them is invisible in development: a singleton holding a scoped `DbContext` behaves perfectly when one request runs at a time; an unpaginated endpoint is fast on thirty rows; a wide-open CORS policy is fine until a real frontend calls you from a real domain. Production is where those become bugs, and production is a bad place to learn them.

Who this is for

Junior-to-mid .NET developers shipping their first production Web API. You should be comfortable writing C# and reading LINQ. You do not need to have deployed anything before. That is rather the point.

If you have shipped several APIs already, treat this as a review checklist rather than a tutorial. The items are short and the code is focused; you will know within a paragraph whether an item is one you already handle.

How to use it

The book is 33 items across seven chapters, and every item has the same three parts:

- **Why it matters:** the concrete failure. Not "best practice", but what goes wrong and when.
- **How to do it right:** a short explanation and code you can paste into a project and run.
- **Go deeper:** where in the [Ultimate ASP.NET Core Web API](#) course the topic gets full treatment, if you want it.

There are two sensible ways to read it. Front to back, in order. The chapters build on each other, and Chapter 1 assumes nothing while Chapter 7 assumes the rest. Or straight to the checklist on the next page, tick off what you already do, and read only the items you could not tick.

Either way, the checklist page is the one to print. Run it against your API before a release the way a pilot runs a pre-flight list: not because you have forgotten how to fly, but because the one item you skipped is always the one that mattered.

About the code

Every item has runnable code, never prose-only advice. The samples target **.NET 10** and current idioms: minimal hosting, nullable reference types enabled, primary constructors where they read better, collection expressions where they fit.

The examples follow the conventions of the [Ultimate ASP.NET Core Web API](#) course, so the code here and the code there are the same shape. That means a layered solution (domain and services in the core, EF Core behind repository abstractions, controllers in their own presentation assembly) and a running example of companies and their employees. `Company`, `Employee`, `CompanyDto`, and `EmployeeForCreationDto` appear throughout. You do not have to adopt that structure to use the advice; where an item depends on it, the text says so.

The samples are deliberately focused. They show the thing being discussed and leave out the surrounding scaffolding you already know how to write, so nothing is padded to look like a complete project.

The checklist

The whole book on one page. Print it, run it against your API before a release, and read the item behind anything you cannot tick.

CHAPTER 1: FOUNDATIONS

- ☐ **1.1** Controllers sit behind a service layer, never straight on top of `DbContext`.
- ☐ **1.2** Every service is registered with the lifetime it actually needs, and no singleton captures a scoped dependency.
- ☐ **1.3** No entity is ever returned from an action. Read, creation, and update DTOs are separate types.
- ☐ **1.4** The middleware pipeline is registered in a deliberate order: exceptions first, forwarded headers early, authentication before authorization.

CHAPTER 2: CONTRACTS & VALIDATION

- ☐ **2.1** Every incoming payload is validated, including the non-nullable value types that quietly bind to `0`.
- ☐ **2.2** The validation prologue lives in an action filter, not copy-pasted into every write action.
- ☐ **2.3** Rules that outgrow attributes (conditional, cross-field) move to `FluentValidation`, and one engine is used per project.
- ☐ **2.4** Status codes are correct and deliberate: `201` with a `Location` header, `204` on delete, `400` versus `422`, never `200` with an error body.
- ☐ **2.5** PATCH is real JSON Patch, and the patched object is re-validated before it is saved.
- ☐ **2.6** GET, HEAD, and OPTIONS have no side effects; PUT is a full replacement; DELETE twice is not an error.

CHAPTER 3: ERROR HANDLING

- ☐ **3.1** There are zero `try/catch` blocks in the controllers. One `ExceptionHandler` catches everything.
- ☐ **3.2** Errors come back as RFC 7807 problem details with the `application/problem+json` content type, and a 500 body leaks nothing.
- ☐ **3.3** Expected failures are typed exceptions mapped to status codes, not `null` returns.

CHAPTER 4: SECURITY

- ☐ **4.1** All four JWT validation parameters are on: issuer, audience, lifetime, signing key.
- ☐ **4.2** The signing secret is not in `appsettings.json` and not in the repository, and a missing one stops startup.
- ☐ **4.3** Configuration is bound to typed options with `ValidateOnStart()`, not read through string keys.
- ☐ **4.4** Users go through ASP.NET Core Identity. Nothing hand-rolls password hashing, and lockout is configured.
- ☐ **4.5** Every non-public endpoint carries `[Authorize]`, with `[AllowAnonymous]` used deliberately and visibly.
- ☐ **4.6** Access tokens are short-lived, refresh tokens are cryptographically random, stored, and rotated on every use.
- ☐ **4.7** CORS names its origins, methods, and headers, and exposes `X-Pagination` if clients read it.

CHAPTER 5: PERFORMANCE & SCALE

- ☐ **5.1** No endpoint can return an unbounded collection. The unpaginated request is bounded too.
- ☐ **5.2** The whole chain is async with a flowed `CancellationToken`, and reads run `AsNoTracking()`.
- ☐ **5.3** Filtering, searching, and sorting happen in the database, composed on `IQueryable` before `ToListAsync`.
- ☐ **5.4** Anything a caller can interpolate into a query is whitelisted, and paginated queries are always deterministically ordered.
- ☐ **5.5** Cached responses have an eviction story: tagged policies evicted on write, and ETags derived from real state.
- ☐ **5.6** A rate limiter is configured with an explicit `429`, a `Retry-After` header, and a problem-details body.

CHAPTER 6: OBSERVABILITY & DOCUMENTATION

- ☐ **6.1** Logging is structured with message templates, exceptions are logged as objects, and there is an explicit never-log list.
- ☐ **6.2** Liveness and readiness are separate endpoints, and readiness checks the dependencies that matter.
- ☐ **6.3** The OpenAPI document has an `Authorize` button, XML comments, and a `[ProducesResponseType]` for every status code you promised.

CHAPTER 7: SHIP IT

- ☐ **7.1** The API is versioned from day one, with one version reader chosen and deprecation advertised before removal.
 - ☐ **7.2** Integration tests run through the real pipeline against a real database, covering `401`, `404`, `415`, `422`, and a `201` with a `Location`.
 - ☐ **7.3** Configuration layers per environment, production values live outside the repository, and a missing setting fails at boot.
 - ☐ **7.4** Deployment is a scripted `dotnet publish -c Release`, the host supplies the secrets, and the deploy is verified against a live token and a healthy `/health`.
-

Chapter 1: Foundations

Everything else in this book assumes your project is shaped sensibly. If your controllers talk straight to `DbContext`, your services are registered with the wrong lifetimes, your entities are your contract, and your middleware runs in the wrong order, no amount of caching or rate limiting will save you.

These four items are the ones you want in place before you write your first real endpoint, because retrofitting them later means touching every file you wrote in between.

1.1 Put controllers behind services, not straight on top of your DbContext

Why it matters

A controller that opens a `DbContext` and writes LINQ inline can only be exercised by starting the whole web host. You cannot unit test the rule it encodes, you cannot call it from a background job, and when the same rule is needed by a second endpoint it gets copy-pasted, usually with a small difference nobody notices until a customer reports it. The controller also ends up knowing about EF Core, which means an EF upgrade or a switch to a second data store touches your HTTP layer for no good reason.

The version of this that hurts most in production is subtler: business logic living in the controller can only run when there is an HTTP request. The day you need "the same thing, but on a schedule", you have to reimplement it.

How to do it right

Push the work down. The controller reads the request, hands off to a service, and turns the result into a response. The service holds the business rule and talks to a repository abstraction. The repository is the only thing that knows EF Core exists.

Two aggregating types keep the constructor lists sane: a repository manager that exposes every repository and owns `SaveChangesAsync`, and a service manager that exposes every service. Both create their members lazily, so a request that only touches companies never constructs the employee repository.

```
// Core.Domain/Repositories/IRepositoryManager.cs
public interface IRepositoryManager
{
    ICompanyRepository Company { get; }
    IEmployeeRepository Employee { get; }
    Task SaveAsync(CancellationToken ct = default);
}

// Infrastructure.Persistence/RepositoryManager.cs
```

```

public sealed class RepositoryManager : IRepositoryManager
{
    private readonly RepositoryContext _context;
    private readonly Lazy<ICompanyRepository> _company;
    private readonly Lazy<IEmployeeRepository> _employee;

    public RepositoryManager(RepositoryContext context)
    {
        _context = context;
        _company = new Lazy<ICompanyRepository>(() => new CompanyRepository(context));
        _employee = new Lazy<IEmployeeRepository>(() => new EmployeeRepository(context));
    }

    public ICompanyRepository Company => _company.Value;
    public IEmployeeRepository Employee => _employee.Value;

    public Task SaveAsync(CancellationTokens ct = default) => _context.SaveChangesAsync(ct);
}

```

The service depends on the abstraction, never on the context:

```

// Core.Services/CompanyService.cs
internal sealed class CompanyService : ICompanyService
{
    private readonly IRepositoryManager _repository;
    private readonly IMapper _mapper;

    public CompanyService(IRepositoryManager repository, IMapper mapper)
    {
        _repository = repository;
        _mapper = mapper;
    }

    public async Task<IEnumerable<CompanyDto>> GetAllAsync(CancellationTokens ct)
    {
        var companies = await _repository.Company.GetAllAsync(trackChanges: false, ct);
        return _mapper.Map<IEnumerable<CompanyDto>>(companies);
    }
}

```

And the controller shrinks to routing plus a response:

```

// Infrastructure.Presentation/Controllers/CompaniesController.cs
[Route("api/companies")]
[ApiController]
public class CompaniesController(IServiceManager service) : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult> GetCompanies(CancellationTokens ct)
        => Ok(await service.CompanyService.GetAllAsync(ct));
}

```

One extra move is worth the effort: put the controllers in their own class library that references only the service abstractions, and load it from the host with `AddApplicationPart`. Now a controller physically cannot inject a `DbContext`, because the project it lives in does not reference persistence at all.

```
// Program.cs
builder.Services.AddControllers()
    .AddApplicationPart(typeof(Presentation.AssemblyReference).Assembly);
```

The class library needs `<FrameworkReference Include="Microsoft.AspNetCore.App" />` in its `.csproj` to see `ControllerBase`, and `AssemblyReference` is just an empty static class you point the host at.

Go deeper: the *Onion Architecture Implementation* chapter of Ultimate ASP.NET Core Web API builds this layering project by project, from the repository base class through to the presentation assembly.

1.2 Register every service with the lifetime it actually needs

Why it matters

Lifetimes are the quietest bug in ASP.NET Core. A singleton that captures a scoped `DbContext` works perfectly on your machine, where one request runs at a time. Under real concurrency, two requests share one context, one request's `SaveChanges` writes the other's changes, and the tracked entity graph slowly corrupts. Nothing throws; you just get wrong data.

The reverse mistake is cheaper but still real: registering something expensive as transient means you build it on every injection point, several times per request.

How to do it right

Three lifetimes, three rules:

- **Transient:** a new instance every time it is resolved. Use it for cheap, stateless helpers.
- **Scoped:** one instance per HTTP request, shared by everything in that request. This is where anything touching the database belongs, `DbContext` included.
- **Singleton:** one instance for the life of the process. Only for genuinely stateless or thread-safe things: a logger wrapper, a cache client, an in-memory lookup.

The rule that prevents the corruption bug: **a service can only depend on something with an equal or longer lifetime**. A singleton must never hold a scoped dependency. If a singleton truly needs a scoped service, inject `IServiceScopeFactory` and create a scope per unit of work.

Keep the registrations in extension methods on `IServiceCollection` so `Program.cs` stays readable:

```
// Extensions/ServiceExtensions.cs
public static class ServiceExtensions
{
    public static void ConfigureRepositoryManager(this IServiceCollection services)
        => services.AddScoped<IRepositoryManager, RepositoryManager>();

    public static void ConfigureServices(this IServiceCollection services)
        => services.AddScoped<IServiceManager, ServiceManager>();

    public static void ConfigureLoggerService(this IServiceCollection services)
        => services.AddSingleton<ILoggerManager, LoggerManager>();

    public static void ConfigureSqlContext(this IServiceCollection services, IConfiguration
    config)
        => services.AddDbContext<RepositoryContext>(opts =>
            opts.UseSqlServer(config.GetConnectionString("sqlConnection")));
}
```

```
// Program.cs
builder.Services.ConfigureLoggerService();
builder.Services.ConfigureSqlContext(builder.Configuration);
builder.Services.ConfigureRepositoryManager();
builder.Services.ConfigureServiceManager();
```

`AddDbContext` registers scoped by default, and the repository and service managers are scoped to match. A matching chain, no capture.

When you have two implementations of one interface and need both, do not rely on registration order (last one wins, which is a trap). Use keyed services:

```
builder.Services.AddKeyedScoped<IReportExporter, CsvExporter>("csv");
builder.Services.AddKeyedScoped<IReportExporter, PdfExporter>("pdf");

public class ReportsController(
    [FromKeyedServices("pdf")] IReportExporter exporter) : ControllerBase
{
    // ...
}
```

One safety net worth turning on: in development, the container validates scopes and throws at startup if a singleton captures a scoped service. `WebApplication.CreateBuilder` does this in the Development environment already, which is exactly why you should never skip running the app in Development before you ship.

Go deeper: the *Dependency Injection in ASP.NET Core* chapter walks each lifetime with a live example, and finishes on keyed services.

1.3 Never expose your entities; return DTOs

Why it matters

Your entity is a database shape, not an API contract. Return it directly and you publish whatever is on it: the internal `IsDeleted` flag, the `PasswordHash`, the audit columns, the navigation property that drags a thousand children into the response. Serializers do not know which fields you meant to be public.

The second cost is worse over time. Once entities are the contract, every schema change is a breaking API change. Rename a column for clarity and you break every client. Add a relationship and your payload silently doubles in size. A DTO layer decouples the two, so you can refactor the database without a version bump.

How to do it right

Define read DTOs as records in a shared project, and keep them free of navigation properties. Concatenate, flatten, or drop whatever the client does not need:

```
// Shared/DataTransferObjects/CompanyDto.cs
namespace Shared.DataTransferObjects;

public record CompanyDto(Guid Id, string Name, string FullAddress);
```

Records are a good fit here: immutable by default, value equality, and a one-line declaration. Positional syntax is fine for read DTOs, which carry no attributes.

Write DTOs as separate types, not the same type reused. A creation payload must not accept an `Id` the server owns, and an update payload may legitimately require fields a read model computes. Once validation attributes come in (Chapter 2), positional records get hard to read, so switch those to init-only properties:

```
// Shared/DataTransferObjects/CompanyForCreationDto.cs
public record CompanyForCreationDto
{
    [Required(ErrorMessage = "Company name is a required field.")]
    [MaxLength(60, ErrorMessage = "Maximum length for the Name is 60 characters.")]
    public string? Name { get; init; }

    [Required(ErrorMessage = "Address is a required field.")]
    public string? Address { get; init; }

    public string? Country { get; init; }
}
```

Map between entity and DTO with AutoMapper in a profile class. Mapping into a positional record needs `ForCtorParam`, not `ForMember`. `ForMember` fails at runtime because the record has no parameterless constructor to set properties on:

```
// MappingProfile.cs
public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<Company, CompanyDto>()
            .ForMember(nameof(CompanyDto.FullAddress),
                opt => opt.MapFrom(c => $"{c.Address} {c.Country}"));

        CreateMap<CompanyForCreationDto, Company>();
    }
}
```

```
dotnet add package AutoMapper --version 13.0.1
```

```
// Program.cs
builder.Services.AddAutoMapper(typeof(MappingProfile));
```

Version 13.0.1 is pinned deliberately. AutoMapper 15.0 and later require a commercial licence key and throw at startup without one; the course source is built against 13.0.1.

Do the mapping in the service layer, so the controller never sees an entity type and cannot accidentally return one.

Manual mapping is perfectly respectable, and faster. The reason to reach for a mapper is repetition, not performance. Either way, the rule is the same: nothing from your domain assembly appears in a response body.

Go deeper: *DTO Classes vs. Entity Model Classes and Using AutoMapper in ASP.NET Core*, in the Handling GET Requests chapter, cover the split and the `ForMember` gotcha.

1.4 Get your middleware order right, and know what each one does

Why it matters

The request pipeline runs top to bottom, in the order you registered it. Get the order wrong and you get failures that never show up in development. `UseCors` in the wrong place works locally, where the browser and API share an origin, and fails the moment a real frontend on a different domain calls you. `UseAuthorization` before `UseAuthentication` means there is no identity to authorize when the check runs, so every protected endpoint 401s. Miss `UseForwardedHeaders` behind a reverse proxy and every client IP in your logs is the proxy's, and every generated URL says `http` when the world sees `https`.

None of these produce a helpful error. They produce "it worked on staging".

How to do it right

Treat the pipeline as one ordered block and register it deliberately. Exception handling goes first, so it wraps everything after it. Anything that shapes the request, such as forwarded headers, must run before anything that reads it. Authentication before authorization, always. CORS before anything that could short-circuit a response the browser needs headers on.

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.ConfigureCors();
builder.Services.ConfigureLoggerService();
builder.Services.ConfigureSqlContext(builder.Configuration);
builder.Services.ConfigureRepositoryManager();
builder.Services.ConfigureServiceManager();
builder.Services.AddControllers()
    .AddApplicationPart(typeof(Presentation.AssemblyReference).Assembly);

var app = builder.Build();

// 1. Exception handling first, so it wraps everything below.
app.UseExceptionHandler();

// 2. Trust X-Forwarded-* from your proxy so scheme and client IP are real.
app.UseForwardedHeaders(new ForwardedHeadersOptions
{
    ForwardedHeaders = ForwardedHeaders.XForwardedFor
        | ForwardedHeaders.XForwardedProto
        | ForwardedHeaders.XForwardedHost
});

// 3. HSTS in non-development only; it is cached hard by browsers.
if (!app.Environment.IsDevelopment())
    app.UseHsts();

app.UseHttpsRedirection();

// 4. Rate limiting before CORS, so rejections still carry CORS headers.
app.UseRateLimiter();

// 5. CORS before caching and before the endpoints it protects.
app.UseCors("CorsPolicy");

app.UseResponseCaching();

// 6. Who are you, then what may you do. Never the other way round.
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
```

Two of these orderings are easy to get wrong and worth memorising:

- **UseCors** before **UseResponseCaching**. Otherwise a response can be cached without its CORS headers and then served to an origin it should not reach.
- **UseRateLimiter** before **UseCors**. A 429 that lacks CORS headers is invisible to the browser. The client sees an opaque network error instead of "you are being throttled".

Middleware is just a chain of delegates. Each component can do work, call `next`, and then do more work as the response unwinds:

```
app.Use(async (context, next) =>
{
    var stopwatch = Stopwatch.StartNew();
    await next.Invoke();           // everything below this line runs here
    stopwatch.Stop();
    app.Logger.LogInformation("{Method} {Path} -> {Status} in {Ms}ms",
        context.Request.Method, context.Request.Path,
        context.Response.StatusCode, stopwatch.ElapsedMilliseconds);
});
```

One rule about writing to the response: once you have written the body, do not call `next`. The component after you will try to set a status code or a header on a response that has already started, and you will get an exception at the least convenient moment.

Go deeper: the *Project Configuration* chapter's *ASP.NET Core Middleware* section walks the pipeline order diagram, `Use` / `Run` / `Map`, and short-circuiting.

What this chapter had to leave out

- Building the onion solution project by project (entities, the context class and connection, Fluent API configuration, migrations and the initial data seed, the repository base class, and the `RepositoryManager` unit of work) across eleven lessons rather than three code samples.
- Why the presentation assembly needs `AddApplicationPart`, what the error looks like when you forget it, and how the controllers get discovered once you have.
- Keyed services, and when resolving a dependency by name is the right answer instead of inventing a second interface.
- The Serilog logging service built from an empty project through to a Seq sink, where structured logs stop being text and start being queryable.
- Routing, route constraints, route binding, and resource naming: the conventions every endpoint you write afterwards inherits.

The full onion build, project by project, is Chapter 6 of the course. [Get Ultimate ASP.NET Core Web API](#)

Chapter 2: Contracts & Validation

Your API's contract is everything the outside world can rely on: which payloads you accept, which status codes you return, and what each HTTP verb promises. Get it wrong and you do not get a compiler error. You get a 500 where a 422 belonged, a client that cannot tell success from failure, and a retry loop that quietly duplicates rows. The six items in this chapter lock the contract down before anyone else depends on it.

2.1 Validate every incoming payload; never trust the client

Why it matters

Everything arriving in a request body is a string the caller chose. If you do not check it, a `null` name or a 5,000-character position travels through your mapper and into EF Core, where it fails as a `DbUpdateException` and reaches the caller as a `500 Internal Server Error`. That status is a lie: nothing went wrong on the server, the request was bad. The caller cannot tell whether to fix the payload or retry, and your on-call alerting fires for a client mistake.

How to do it right

Put the rules on the DTO you deserialize into, not on the entity, and not in the action. Because the creation and update DTOs almost always share the same rules, declare them once on an abstract base record and inherit. Use init-only properties rather than positional record parameters; once attributes pile up, positional syntax stops being readable.

```

using System.ComponentModel.DataAnnotations;

namespace Shared.DataTransferObjects;

public abstract record EmployeeForManipulationDto
{
    [Required(ErrorMessage = "Employee name is a required field.")]
    [MaxLength(30, ErrorMessage = "Maximum length for the Name is 30 characters.")]
    public string? Name { get; init; }

    // int is never null. Without a Range, a missing "age" binds to 0 and passes.
    [Range(18, int.MaxValue, ErrorMessage = "Age is required and cannot be lower than 18.")]
    public int Age { get; init; }

    [Required(ErrorMessage = "Position is a required field.")]
    [MaxLength(20, ErrorMessage = "Maximum length for the Position is 20 characters.")]
    public string? Position { get; init; }
}

public record EmployeeForCreationDto : EmployeeForManipulationDto;
public record EmployeeForUpdateDto : EmployeeForManipulationDto;

```

That comment on `Age` is the trap everybody hits. `[Required]` on a value type does nothing useful: model binding fills the missing property with `default`, and `0` is not null, so validation passes and an infant employee lands in your database. Any non-nullable value type needs a range, not a required flag.

Model binding runs before your action, and validation runs right after it, so by the time the action body executes, `ModelState` already knows the answer:

```

[HttpPost]
public async Task<IActionResult> CreateEmployeeForCompany(Guid companyId,
    [FromBody] EmployeeForCreationDto employee, CancellationToken ct)
{
    if (employee is null)
        return BadRequest("EmployeeForCreationDto object is null.");

    if (!ModelState.IsValid)
        return UnprocessableEntity(ModelState);

    var created = await _service.EmployeeService
        .CreateEmployeeForCompanyAsync(companyId, employee, trackChanges: false, ct);

    return CreatedAtRoute("GetEmployeeForCompany",
        new { companyId, id = created.Id }, created);
}

```

Validate input, never output. GET responses do not need rules, because you produced that data. And when built-in attributes run out, you have two ways forward: a custom `ValidationAttribute` when the rule is reusable across models, or `IValidatableObject` on the model when the rule spans several properties of that one type and will never be reused.

Go deeper: the course module *Validation in ASP.NET Core Web API* walks the built-in attributes, custom attributes, and `IDataValidatableObject` in full.

2.2 Move validation out of your actions into a filter

Why it matters

Look at the top of every POST and PUT action and you will find the same four lines: null check, `ModelState` check, two early returns. Copy-pasted prologues rot. Somebody adds an endpoint at 6pm, skips the `ModelState` check, and that one action now hands invalid data to the service layer while every other action rejects it. The bug is invisible in review because the action looks like all the others, only two lines shorter.

How to do it right

Extract the prologue into an action filter. Filters hook into the MVC invocation pipeline, and `OnActionExecuting` runs after model binding and validation but before your action body, exactly the window you need. Setting `context.Result` short-circuits the pipeline, so the action never runs.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace CompanyEmployees.Infrastructure.Presentation.ActionFilters;

public sealed class ValidationFilterAttribute : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        var action = context.RouteData.Values["action"];
        var controller = context.RouteData.Values["controller"];

        var param = context.ActionArguments
            .SingleOrDefault(x => x.Key.EndsWith("Dto", StringComparison.OrdinalIgnoreCase)
                || x.Value?.GetType().Name.Contains("Dto") == true).Value;

        if (param is null)
        {
            context.Result = new BadRequestObjectResult(
                $"Object is null. Controller: {controller}, action: {action}");
            return;
        }

        if (!context.ModelState.IsValid)
            context.Result = new UnprocessableEntityObjectResult(context.ModelState);
    }

    public void OnActionExecuted(ActionExecutedContext context) { }
}
```

Register it in the container and apply it where it belongs:

```
builder.Services.AddScoped<ValidationFilterAttribute>();
```

```
[HttpPost]
[ServiceFilter(typeof(ValidationFilterAttribute))]
public async Task<IActionResult> CreateCompany(
    [FromBody] CompanyForCreationDto company, CancellationToken ct)
{
    var created = await _service.CompanyService.CreateCompanyAsync(company, ct);
    return CreatedAtRoute("CompanyById", new { id = created.Id }, created);
}
```

Two details matter. Use `[ServiceFilter]`, not `[TypeFilter]` or a bare attribute, when the filter needs constructor-injected dependencies. `[ServiceFilter]` resolves it from the container, which is why the `AddScoped` registration is required. And know the scopes: filters run global first, then controller, then action, and you can override that with the `Order` property when two filters must run in a specific sequence.

The action is now three lines and says only what it does. Every write endpoint gets identical validation behaviour for free, and there is exactly one place to change it.

Go deeper: the course module *Action Filters in ASP.NET Core Web API* covers filter types, scope, and invocation order before building this exact filter.

2.3 Reach for FluentValidation when rules outgrow attributes

Why it matters

Data Annotations handle "not empty" and "under 30 characters" well. They handle "discount is only allowed when the order total exceeds 100 and the customer is not on a trial plan" badly. You end up writing a custom attribute that reaches into `validationContext.ObjectInstance` and casts, or an `IValidatableObject` implementation welded into the model and untestable on its own. Once conditional and cross-field rules appear, attribute soup on a DTO is harder to read than the rules it encodes.

How to do it right

FluentValidation puts each DTO's rules in its own class, which you can unit test without spinning up MVC. Install the core library and its DI extensions in the project holding your controllers. Do not install `FluentValidation.AspNetCore`; that package is deprecated and no longer maintained:

```
dotnet add package FluentValidation --version 12.1.1
dotnet add package FluentValidation.DependencyInjectionExtensions --version 12.1.1
```

Write a validator per DTO. Better still, since creation and update usually share rules, write one generic base and thin subclasses:

```
using FluentValidation;
using Shared.DataTransferObjects;

public abstract class EmployeeForManipulationDtoValidator<T> : AbstractValidator<T>
    where T : EmployeeForManipulationDto
{
    protected EmployeeForManipulationDtoValidator()
    {
        RuleFor(e => e.Name)
            .NotEmpty().WithMessage("Employee name is a required field.")
            .MaximumLength(30).WithMessage("Maximum length for the Name is 30 characters.");

        RuleFor(e => e.Age)
            .GreaterThanOrEqualTo(18).WithMessage("Age can't be lower than 18.");

        RuleFor(e => e.Position)
            .NotEmpty().WithMessage("Position is a required field.")
            .MaximumLength(20).WithMessage("Maximum length for the Position is 20 characters.");

        // The kind of rule attributes cannot express:
        RuleFor(e => e.Position)
            .Must(p => p != "Director").When(e => e.Age < 25)
            .WithMessage("Directors must be at least 25.");
    }
}

public class EmployeeForCreationDtoValidator
    : EmployeeForManipulationDtoValidator<EmployeeForCreationDto>;

public class EmployeeForUpdateDtoValidator
    : EmployeeForManipulationDtoValidator<EmployeeForUpdateDto>;
```

Register every validator in the assembly once:

```
builder.Services
    .AddValidatorsFromAssemblyContaining(typeof(EmployeeForCreationDtoValidator));
```

Then run it explicitly. Inject `IValidator<T>` per action with `[FromServices]` so the constructor does not grow a parameter for every DTO the controller touches:

```
[HttpPost]
public async Task<IActionResult> CreateEmployeeForCompany(Guid companyId,
    [FromBody] EmployeeForCreationDto employee,
    [FromServices] IValidator<EmployeeForCreationDto> validator,
    CancellationToken ct)
{
    if (employee is null)
        return BadRequest("EmployeeForCreationDto object is null.");

    var result = await validator.ValidateAsync(employee, ct);
    if (!result.IsValid)
        return UnprocessableEntity(result.ToDictionary());

    var created = await _service.EmployeeService
        .CreateEmployeeForCompanyAsync(companyId, employee, trackChanges: false, ct);

    return CreatedAtRoute("GetEmployeeForCompany",
        new { companyId, id = created.Id }, created);
}
```

`ToDictionary()` produces the same property-to-errors shape `ModelState` gives you, so clients see one error format regardless of which engine produced it.

Explicit invocation is deliberate. FluentValidation's older auto-validation integration (the `FluentValidation.AspNetCore` package) ran validators during model binding and folded errors into `ModelState`; it is deprecated and it hides where validation happens. Calling `ValidateAsync` yourself keeps the flow readable and lets you validate objects your action built rather than only those it bound. Pick one engine per project. Mixing attributes and validators on the same DTO means two places to look when a rule surprises you.

Go deeper: the *Fluent Validation in Web API* lesson in the course's validation module builds this out as an alternative to the Data Annotations path used elsewhere.

2.4 Return the right status code for the right situation

Why it matters

`200 OK` with `{"error": "..."} in the body forces every client to parse the body before it knows whether the call worked, and generated clients, retry policies, and monitoring dashboards will all read the 200 and move on. The mirror-image mistake is 400 Bad Request for a request that parsed perfectly and only failed a business rule. Status codes are the part of your contract that infrastructure reads. Get them wrong and caching, retries, and alerting all make the wrong decision.`

How to do it right

Learn the small set you actually need for CRUD:

Situation	Status	Notes
GET succeeded	200 OK	Body carries the resource
POST created a resource	201 Created	Location header points at the new resource
PUT/PATCH/DELETE succeeded, nothing to return	204 No Content	Empty body
Malformed body, unparseable JSON, null payload	400 Bad Request	The request never became a model
Well-formed body, failed validation rules	422 Unprocessable Entity	Syntax fine, semantics wrong
Caller not authenticated / not permitted	401 / 403	See Chapter 4
Resource does not exist	404 Not Found	
Unhandled failure	500 Internal Server Error	Your fault, never the caller's

The 400-versus-422 split is the one worth defending. 400 says "I could not understand this request." 422 says "I understood it and it broke my rules." A client can act differently on those two, which is the whole point.

Use CreatedAtRoute for POST so the caller gets a URL rather than having to construct one:

```
[HttpGet("{id:guid}", Name = "CompanyById")]
[ProducesResponseType(StatusCodes.Status200OK, Type = typeof(CompanyDto))]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> GetCompany(Guid id, CancellationToken ct)
    => Ok(await _service.CompanyService.GetCompanyAsync(id, trackChanges: false, ct));

[HttpPost]
[ServiceFilter(typeof(ValidationFilterAttribute))]
[ProducesResponseType(StatusCodes.Status201Created, Type = typeof(CompanyDto))]
[ProducesResponseType(StatusCodes.Status422UnprocessableEntity)]
public async Task<IActionResult> CreateCompany(
    [FromBody] CompanyForCreationDto company, CancellationToken ct)
{
    var created = await _service.CompanyService.CreateCompanyAsync(company, ct);
    return CreatedAtRoute("CompanyById", new { id = created.Id }, created);
}
```

Now the catch that confuses everyone. [ApiController] automatically converts an invalid ModelState into a 400 before your action runs. Your breakpoint on the first line never hits, and your carefully chosen 422 never happens. If you want to own that response, suppress the behaviour:

```
builder.Services.Configure<ApiBehaviorOptions>(options =>
{
    options.SuppressModelStateInvalidFilter = true;
});
```

Keep `[ApiController]` itself, because you still want attribute routing enforcement, binding source inference, and problem-details formatting for error codes. You are only turning off the automatic 400.

Return `ActionResult` when an action has several outcomes, `ActionResult<T>` when you want the payload type inferred, and always add `[ProducesResponseType]` so your OpenAPI document tells the truth about what callers can expect (see item 6.3).

Go deeper: the course covers this across *Web API Return Types*, *Handling POST Requests*, and *Validation From the ApiController Attribute*.

2.5 Support PATCH properly if you claim to support it

Why it matters

Developers reach for PATCH expecting "send me only the fields that changed." If your PATCH endpoint binds a full DTO and maps it onto the entity, every field the caller omitted arrives as `null` or `0` and overwrites good data. The caller updates a job title and silently wipes a salary. That is data loss, and it will be reported as "the API deleted my data," because it did.

How to do it right

Use JSON Patch (RFC 6902). The request body is an array of operations, not a resource:

```
[
  { "op": "replace", "path": "/position", "value": "Senior Developer" }
]
```

The media type is `application/json-patch+json`. Since .NET 10, JSON Patch runs on `System.Text.Json`. Older guides tell you to pull in `Newtonsoft.Json` and register a separate patch input formatter; that workaround is gone. Install one package in the project holding your controllers, and nothing changes in `Program.cs`:

```
dotnet add package Microsoft.AspNetCore.JsonPatch.SystemTextJson --version 10.0.12
```

Every endpoint, patch documents included, now goes through the same serializer, so your `JsonSerializerOptions` apply everywhere.

The action fetches the entity, maps it to the update DTO, applies the operations to that DTO, then saves:

```
[HttpPatch("{id:guid}")]
public async Task<IActionResult> PartiallyUpdateEmployeeForCompany(Guid companyId, Guid id,
    [FromBody] JsonPatchDocument<EmployeeForUpdateDto> patchDoc, CancellationToken ct)
{
    if (patchDoc is null)
        return BadRequest("patchDoc object sent from client is null.");

    var result = await _service.EmployeeService.GetEmployeeForPatchAsync(
        companyId, id, compTrackChanges: false, empTrackChanges: true, ct);

    // Bad paths and malformed operations land in ModelState.
    patchDoc.ApplyTo(result.employeeToPatch, error =>
        ModelState.AddModelError(error.AffectedObject.GetType().Name, error.ErrorMessage));

    // Re-validate the PATCHED object, not the patch document.
    TryValidateModel(result.employeeToPatch);

    if (!ModelState.IsValid)
        return UnprocessableEntity(ModelState);

    await _service.EmployeeService.SaveChangesForPatchAsync(
        result.employeeToPatch, result.employeeEntity, ct);

    return NoContent();
}
```

Both validation steps are load-bearing, and the second one is the step people skip. The error callback on `ApplyTo` catches an operation against a path that does not exist; without it, `ApplyTo` throws. But a *valid* operation can still produce an *invalid* object: `{"op": "remove", "path": "/age"}` is a perfectly legal operation that sets `Age` to `0`, which your `[Range(18, ...)]` rule forbids. Nothing has validated the object at that point, so `TryValidateModel` re-runs the DTO's rules against the patched instance before it reaches the database.

Go deeper: the course modules *Partially Updating Resources* and the *Validation of PATCH Requests* lesson build this end to end, including the remove-operation trap.

2.6 Make your verbs safe and idempotent the way HTTP says they are

Why it matters

HTTP verbs carry promises that everything between your API and your caller relies on. A method is **safe** if calling it does not change the resource. It is **idempotent** if calling it five times leaves the same state as calling it once. Browsers prefetch safe methods. Proxies and CDNs cache them. HTTP clients, service meshes, and message-driven retries replay idempotent ones automatically after a timeout. Break the promise and you get a GET that deletes records because a link prefetcher followed it, or a duplicated order because a client's socket dropped after your PUT succeeded and the retry policy fired.

Method	Safe	Idempotent
GET	Yes	Yes
HEAD	Yes	Yes
OPTIONS	Yes	Yes
PUT	No	Yes
DELETE	No	Yes
POST	No	No
PATCH	No	No

How to do it right

Each row is an obligation, not a description.

GET/HEAD/OPTIONS must not mutate. No "increment view count" inside a GET, no lazily-created rows, no side-effecting audit writes. If a read genuinely has to record something, put the write on a background channel so the request itself stays safe. HEAD is free if you say so: it is a GET with the body dropped, headers and all:

```
[HttpGet]
[HttpHead] // same action, same headers, no body
public async Task<IActionResult> GetEmployeesForCompany(Guid companyId,
    [FromQuery] EmployeeParameters parameters, CancellationToken ct)
{
    var result = await _service.EmployeeService
        .GetEmployeesAsync(companyId, parameters, trackChanges: false, ct);

    Response.Headers["X-Pagination"] = JsonSerializer.Serialize(result.Metadata);
    return Ok(result.Employees);
}

[HttpOptions]
public IActionResult GetCompaniesOptions()
{
    Response.Headers.Allow = "GET, OPTIONS, POST, PUT, DELETE";
    return Ok();
}
```

PUT must be a full replacement. If PUT merges the fields the caller sent and leaves the rest alone, then two identical PUTs against different starting states produce different results. That is not idempotent, and it is what PATCH is for. Replace the whole resource, every time.

DELETE must be idempotent, which means deleting twice is not an error. The second call finds nothing and that is the correct outcome, because the caller's intent ("this must not exist") is already satisfied:

```
[HttpDelete("{id:guid}")]
public async Task<IActionResult> DeleteCompany(Guid id, CancellationToken ct)
{
    await _service.CompanyService.DeleteCompanyAsync(id, trackChanges: false, ct);
    return NoContent(); // 204 whether or not it existed a moment ago
}
```

Returning `404` on the second DELETE is defensible and common, but it makes a retried delete look like a failure to a client that cannot tell a lost response from a missing resource. `204` is the safer contract.

POST is neither, so protect it yourself. POST creates something new on every call. That is its job, and a retried POST really does create a duplicate. When duplicates are expensive (payments, orders), accept a client-supplied idempotency key, store it with the created resource id, and return the original `201` result if you see the same key again. And keep POST as the only creation verb: if you let PUT create resources at caller-chosen URLs, you have to defend the uniqueness of those URLs forever.

Go deeper: the course module *Method Safety And Method Idempotency* lays out the standard, and *Working With OPTIONS and HEAD Requests* implements the two verbs most APIs forget.

What this chapter had to leave out

- Custom validation attributes and `IValidatableObject`, for the rules that need more than a `[Range]` and a `[Required]`.
- Validation of PUT and validation of PATCH handled separately, because they do not behave the same: the PATCH path has to re-validate a document somebody else wrote.
- Creating a child resource, creating children together with a parent in one request, and creating a whole collection, each with the model binding and `CreatedAtRoute` plumbing that goes with it.
- Deleting a parent together with its children, and the transaction that keeps the two consistent when the second delete fails.
- Content negotiation and a hand-written output formatter, plus data shaping, HATEOAS links, a root document, and custom vendor media types: the REST maturity this book skipped on purpose.

The course spends nine lessons on validation alone, and builds every one of these end to end. [Get Ultimate ASP.NET Core Web API](#)

Chapter 3: Error Handling

Every API fails. What separates a production API from a demo is that the failures come out the same shape every time, carry no stack traces to the caller, and land in your logs with enough detail to diagnose. These three items get you there: one place that catches everything, one response format that clients can parse, and a way to express expected failures without magic nulls.

3.1 Handle exceptions globally; delete every try/catch in your controllers

Why it matters

Per-action `try/catch` blocks drift. You write a good one in `GetCompanies`, copy a worse one into `GetCompany`, and six months later the action somebody added last week has none at all, so an unhandled `SqlException` walks straight through the pipeline and the developer exception page hands a stack trace, a file path, and your ORM version to the internet. Even when every block exists, they disagree: one returns a 500 with a string, another a 400 with an anonymous object, a third swallows the exception and returns an empty 200. Clients cannot code against that, and neither can you.

There is exactly one correct number of exception-handling blocks in your controllers, and it is zero.

How to do it right

Since .NET 8 the answer is `ExceptionHandler`. You implement one method, register the class in DI, and add `UseExceptionHandler` to the pipeline. Because it is a registered service, you inject whatever you need (a logger, configuration, a metrics client) instead of resolving it out of the container by hand.

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Http;

namespace CompanyEmployees;

public sealed class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;

    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger) => _logger = logger;

    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext,
        Exception exception,
        CancellationToken cancellationToken)
    {
        // Log the exception object, not exception.Message. You want the stack trace.
        _logger.LogError(exception, "Unhandled exception for {Method} {Path}",
            httpContext.Request.Method, httpContext.Request.Path);
    }
}
```

```

        httpContext.Response.StatusCode = StatusCodes.Status500InternalServerError;

        await httpContext.Response.WriteAsJsonAsync(new
        {
            title = "An error occurred",
            status = httpContext.Response.StatusCode
        }, cancellationToken);

        return true; // true means: handled, stop the pipeline here.
    }
}

```

Register it, and wire the middleware:

```

builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();

var app = builder.Build();

app.UseExceptionHandler(opt => { }); // empty lambda: the registered handlers do the work

```

Two details matter. Returning `true` tells ASP.NET Core the exception is dealt with and no further handler runs. On .NET 10 it also suppresses the framework's own exception diagnostics, so a handler that returns `true` without logging loses the exception entirely; log before you return. Return `false` and the next registered handler gets a chance, which is how you chain a specific handler (say, one for `DbUpdateConcurrencyException`) in front of a catch-all. And you must remove `UseDeveloperExceptionPage` from your development branch, because it sits earlier in the pipeline and will happily beat your handler to the response.

You may still meet the older form built on `UseExceptionHandler(appError => appError.Run(...))` and `ExceptionHandlerFeature`. It works on every .NET version and there is nothing wrong with the code you have inherited, but it forces you to resolve services manually before `app.Run()` and gives you nowhere clean to add a second handler. On .NET 10, start with `ExceptionHandler`.

The response body above is deliberately thin. The next item replaces it with something standard.

Go deeper: *Global Error Handling* → "Using the `ExceptionHandler` Interface" in the Ultimate ASP.NET Core Web API course walks the refactor from the middleware form to the interface.

3.2 Return RFC 7807 problem details, not ad-hoc error strings

Why it matters

"Internal server error" as a bare string gives a client nothing. There is no field to branch on, no code to map to a translated message, nothing to correlate with your logs when the user opens a support ticket. Worse, most APIs are inconsistent about it: framework-generated errors (a failed model binding, a `NotFound()` from a controller) already come back as RFC 7807 problem details, while your hand-written error path returns some other shape with a different content type. A client now needs two parsers for one API.

RFC 7807 fixes the shape once: `type`, `title`, `status`, `detail`, `instance`, plus any extension members you add. It is served as `application/problem+json`, so the client knows what it is reading before it parses it.

How to do it right

Register `AddProblemDetails()` and write through `IProblemDetailsService` rather than serializing a `ProblemDetails` object yourself. The service applies the registered customisations, sets the correct content type, and, in the part people miss, adds the `traceId` extension member, which is what lets you match a client's screenshot to a log entry.

```
public sealed class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;
    private readonly IProblemDetailsService _problemDetailsService;

    public GlobalExceptionHandler(
        ILogger<GlobalExceptionHandler> logger,
        IProblemDetailsService problemDetailsService)
    {
        _logger = logger;
        _problemDetailsService = problemDetailsService;
    }

    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext,
        Exception exception,
        CancellationToken cancellationToken)
    {
        _logger.LogError(exception, "Unhandled exception for {Method} {Path}",
            httpContext.Request.Method, httpContext.Request.Path);

        httpContext.Response.StatusCode = StatusCodes.Status500InternalServerError;

        var problemDetails = new ProblemDetails
        {
            Title = "An error occurred",
            Status = httpContext.Response.StatusCode,
            Detail = "The request could not be completed.",
            Type = "https://datatracker.ietf.org/doc/html/rfc9110#section-15.6.1"
        };
    }
}
```

```

var written = await _problemDetailsService.TryWriteAsync(new ProblemDetailsContext
{
    HttpContext = httpContext,
    ProblemDetails = problemDetails,
    Exception = exception
});

if (!written)
    await httpContext.Response.WriteAsJsonAsync(problemDetails, cancellationToken);

return true;
}
}

```

Note what is *not* in the body: `exception.Message`, `exception.StackTrace`, the SQL that failed. A 500 is your bug, not the caller's, so the caller gets a stable shape and a trace id and nothing else. The detail belongs in the log, and item 6.1 covers that side. For the 4xx cases the caller *can* act on, the message is safe and useful; item 3.3 shows how to tell the two apart.

Do not set `Response.ContentType` to `"application/json"` by hand before calling `TryWriteAsync`. Leave it alone and the service writes `application/problem+json`, which is the entire point of the exercise. Keep the fallback branch. `TryWriteAsync` returns `false` when no registered writer can handle the request (an unusual `Accept` header, for instance), and you still owe the client a body.

You can enrich every problem response in one place:

```

builder.Services.AddProblemDetails(options =>
{
    options.CustomizeProblemDetails = ctx =>
    {
        ctx.ProblemDetails.Instance = $"{ctx.HttpContext.Request.Method}
{ctx.HttpContext.Request.Path}";
        ctx.ProblemDetails.Extensions["requestId"] = ctx.HttpContext.TraceIdentifier;
    };
});

```

Go deeper: *Global Error Handling* → "Standardizing Exception Handling With ProblemDetails" covers `IProblemDetailsService` and the content-type difference in detail.

3.3 Model expected failures as typed exceptions, not magic nulls

Why it matters

A service method that returns `null` when a company does not exist pushes the decision onto every caller. Most of them remember the check. One does not, and `NullReferenceException` becomes a 500 for what was really a perfectly ordinary 404: a request for a resource that is not there. Worse, the null

carries no reason: was the id malformed, was the row deleted, did the caller lack permission? All three arrive as the same nothing.

The alternative is not to litter your controllers with `if (x is null) return NotFound()` either. That is the try/catch problem from item 3.1 in a different coat.

How to do it right

Give expected failures a type. Create an abstract base per HTTP outcome and a sealed, specific exception per case. The base is what makes this work, because the handler maps *categories*, so adding a fourth not-found case requires no change to the handler at all.

```
namespace CompanyEmployees.Core.Domain.Exceptions;

public abstract class NotFoundException : Exception
{
    protected NotFoundException(string message) : base(message) { }
}

public abstract class BadRequestException : Exception
{
    protected BadRequestException(string message) : base(message) { }
}

public sealed class CompanyNotFoundException : NotFoundException
{
    public CompanyNotFoundException(Guid companyId)
        : base($"The company with id: {companyId} doesn't exist in the database.") { }
}
```

Throw from the service, where the rule lives:

```
public async Task<CompanyDto> GetCompanyAsync(Guid id, bool trackChanges)
{
    var company = await _repository.Company.GetCompanyAsync(id, trackChanges)
        ?? throw new CompanyNotFoundException(id);

    return _mapper.Map<CompanyDto>(company);
}
```

And map categories to status codes in the one handler you already have:

```

httpContext.Response.StatusCode = exception switch
{
    NotFoundException => StatusCodes.Status404NotFound,
    BadRequestException => StatusCodes.Status400BadRequest,
    - => StatusCodes.Status500InternalServerError
};

var detail = exception switch
{
    NotFoundException or BadRequestException => exception.Message,
    - => "The request could not be completed." // never leak internals on a 500
};

```

That last switch is the important half. Your own exception types carry messages you wrote for the caller, so putting them in `Detail` is safe and helpful. An exception you did not anticipate carries a message you did not write (a connection string fragment, a constraint name, a file path), so it never reaches the response. Log it; do not publish it.

One caveat worth knowing: exceptions are not free. On the cold, exceptional paths this pattern targets (a bad id, a missing row), the cost is negligible, measured in single-digit milliseconds. If a failure is *routine* rather than exceptional (a validation endpoint that rejects most input by design, a hot lookup where misses outnumber hits), returning a result type instead of throwing is the better trade.

Go deeper: *Getting Additional Resources With Relations* → "Handling Invalid Requests in a Service Layer" builds this exception hierarchy; the bonus chapter *Improve Response Flow and Performance With Discriminated Unions* shows the exception-free alternative.

What this chapter had to leave out

- The refactor from exception-handling middleware to `ExceptionHandler`, done step by step, including the intermediate version that quietly swallowed the status code.
- `IProblemDetailsService` wired into the pipeline properly, and the content-type difference that decides whether a client can parse your error at all.
- The full service-layer exception hierarchy across companies, employees, and their relations, rather than the two classes this chapter had room for.
- The bonus module on discriminated unions: the same failure paths modelled as return values instead of exceptions, with the service and controller rewritten both ways so you can compare them side by side.

See the refactor happen instead of reading the result. [Get Ultimate ASP.NET Core Web API](#)

Chapter 4: Security

Security is where a Web API stops being a coding exercise and starts being someone else's problem when you get it wrong. The seven items in this chapter cover the whole chain: proving who is calling, protecting the key that makes that proof possible, storing users safely, deciding what each caller is allowed to do, keeping stolen credentials short-lived, and telling the browser exactly which sites may talk to you. None of them are exotic. They are the defaults you have to change before you ship.

4.1 Authenticate with JWT bearer tokens and validate all four parameters

Why it matters

A bearer token is trusted purely because it carries a valid signature. That means every validation switch you turn off is a door you leave open. Set `ValidateIssuer = false` and your API happily accepts a token minted by a completely different system that happens to share a secret. Set `ValidateLifetime = false` globally and a token stolen last March still works today. Set `ValidateIssuerSigningKey = false` and, depending on the handler and algorithm you allow, you are effectively accepting whatever the caller typed into the payload.

The failure is silent. Everything works in development, the tests pass, and nobody discovers the hole until someone goes looking for it.

How to do it right

Register the JWT bearer scheme once, in an extension method, and turn all four validation flags on. Read the issuer and audience from configuration, and the signing key from the environment (item 4.2). Keep the whole thing out of `Program.cs` so the pipeline stays readable.

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

public static class AuthenticationExtensions
{
    public static IServiceCollection ConfigureJwt(
        this IServiceCollection services, IConfiguration configuration)
    {
        var jwt = configuration.GetSection("JwtSettings");
        var secretKey = Environment.GetEnvironmentVariable("SECRET")
            ?? throw new InvalidOperationException("JWT signing secret is not configured.");

        services.AddAuthentication(opt =>
        {
            opt.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
```

```

        opt.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
    })
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,

            ValidIssuer = jwt["validIssuer"],
            ValidAudience = jwt["validAudience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(secretKey)),

            ClockSkew = TimeSpan.FromSeconds(30)
        };
    });

    return services;
}
}

```

Four flags, four distinct jobs:

Parameter	What it proves
<code>ValidateIssuer</code>	The token came from the authority you named in <code>ValidIssuer</code> .
<code>ValidateAudience</code>	The token was minted <i>for this API</i> , not for a sibling service.
<code>ValidateLifetime</code>	The <code>exp</code> and <code>nbf</code> claims still hold, so the token is neither expired nor early.
<code>ValidateIssuerSigningKey</code>	The signature verifies against your key, so the payload was not edited.

Two details worth knowing. `ClockSkew` defaults to five minutes, which quietly extends every token's real lifetime by five minutes; if you issue five-minute tokens, that doubles them. Tighten it once your servers have reliable time. And remember that a JWT payload is base64, not encryption. It is readable by anyone holding the token, so nothing sensitive goes in a claim.

Then wire the middleware in the right order, authentication before authorization:

```

app.UseAuthentication();
app.UseAuthorization();

```

Go deeper: the *Authentication and Authorization in Web API* module walks each validation parameter individually in the *JWT Configuration* and *Implementing Authentication* lessons.

4.2 Keep the signing secret out of appsettings and out of source control

Why it matters

Your signing key is the whole security model. Anyone who has it can mint a token claiming any username and any role, and your API will accept it as genuine. Every validation flag from item 4.1 will pass, because the signature is real. Commit that key to `appsettings.json` and everyone with repository read access is now an administrator: contributors, the CI runner, anyone who forks the repo, anyone who gets a copy of the source three jobs from now.

Worse, rotating it becomes an emergency rather than a routine. You cannot tell who has seen it, so you have to assume everyone has.

How to do it right

Keep the issuer, the audience, and the expiry in `appsettings.json`, since none of those are secrets. Keep the key itself somewhere the repository cannot reach.

```
"JwtSettings": {
  "validIssuer": "CompanyEmployeesAPI",
  "validAudience": "https://localhost:5001",
  "expires": 5
}
```

In development, use user secrets. They live in your user profile, not the project folder:

```
dotnet user-secrets init
dotnet user-secrets set "SECRET" "<a long random value>"
```

In production, let the host supply it. On Windows, create a machine-level environment variable from an elevated command prompt:

```
setx SECRET "<a long random value>" /M
```

`/M` makes it a system variable rather than a per-user one, so the application pool identity can read it. Under IIS you can also attach it to the site directly: Configuration Editor → `system.webServer/aspNetCore` → environment variables, saved into `ApplicationHost.config`. Either way, the value never touches the repo.

Read it the same way in both environments, and fail loudly if it is missing:

```
var secretKey =
    configuration["SECRET"] // user secrets / env in dev
    ?? Environment.GetEnvironmentVariable("SECRET") // host-supplied in prod
    ?? throw new InvalidOperationException(
        "JWT signing secret is missing. Set the SECRET user secret or environment variable.");

if (Encoding.UTF8.GetByteCount(secretKey) < 32)
    throw new InvalidOperationException(
        "JWT signing secret must be at least 256 bits for HMAC-SHA256.");
```

That throw matters. A missing secret should stop the application at startup, not produce a `null` that becomes a confusing crash on the first login attempt an hour after deployment.

Three rules that follow from this:

- **A secret you can read in a tutorial is not a secret.** Every value printed in a book, a blog post, or a course, including any you see in this one, is public the moment it is published. Generate your own: 32 random bytes, base64-encoded.
- **Rotate on a schedule, and after anyone with access leaves.** Give yourself a way to accept two keys briefly so rotation does not log everybody out at once.
- **A leaked key means every issued token is suspect.** Rotate the key, and the refresh tokens from item 4.6 give you the mechanism to force everyone back through login.

Go deeper: the *JWT Configuration* lesson sets the secret up as a machine environment variable, and *Configuring Environment File* in the *Deployment to IIS* module shows how to hand the same variable to IIS.

4.3 Bind configuration to typed options instead of stringly-typed lookups

Why it matters

`configuration["JwtSettings:Expires"]` does not fail when you misspell it. It returns `null`. That `null` flows onward, `Convert.ToDouble(null)` returns `0`, and now every token you issue expires the instant it is created, in production, at 3am, with nothing in the logs but a wave of 401s.

String keys are also invisible to your compiler and to your IDE. Rename the section in `appsettings.json` and nothing tells you about the six places that still look for the old name.

How to do it right

Define a class that mirrors the section, bind it once, and inject it where it is needed.

```
namespace CompanyEmployees.Core.Domain.ConfigurationModels;

public sealed class JwtConfiguration
{
    public const string Section = "JwtSettings";

    [Required] public string ValidIssuer { get; init; } = default!;
    [Required] public string ValidAudience { get; init; } = default!;
    [Range(1, 60)] public int Expires { get; init; }
}
```

Register it with validation, so a missing or nonsensical value stops startup instead of surfacing later:

```
public static IServiceCollection AddJwtConfiguration(
    this IServiceCollection services, IConfiguration configuration)
{
    services.AddOptions<JwtConfiguration>()
        .Bind(configuration.GetSection(JwtConfiguration.Section))
        .ValidateDataAnnotations()
        .ValidateOnStart();

    return services;
}
```

Then consume it through `IOptions<T>` rather than reaching for `IConfiguration`:

```
internal sealed class AuthenticationService : IAuthenticationService
{
    private readonly JwtConfiguration _jwt;
    private readonly UserManager<User> _userManager;

    public AuthenticationService(
        UserManager<User> userManager, IOptions<JwtConfiguration> options)
    {
        _userManager = userManager;
        _jwt = options.Value;
    }

    private JwtSecurityToken GenerateTokenOptions(
        SigningCredentials credentials, List<Claim> claims) =>
        new(
            issuer: _jwt.ValidIssuer,
            audience: _jwt.ValidAudience,
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(_jwt.Expires),
            signingCredentials: credentials);
}
```

`ValidateOnStart()` is the part that turns a 3am mystery into a deployment that refuses to start. Use it.

There are three flavours of the interface, and the choice is about reloading, not preference:

- `IOptions<T>`: singleton, bound once at registration, never reloads. The right default.

- `IOptionsSnapshot<T>`: scoped, re-reads per request so file edits take effect without a restart. Cannot be injected into a singleton.
- `IOptionsMonitor<T>`: singleton, supports reloading through `CurrentValue`, and can be injected anywhere. Use it when a singleton needs live values.

One catch: inside a service-registration extension like `ConfigureJwt`, the container has not been built yet, so you cannot resolve `IOptions<T>` there. Bind directly with `configuration.GetSection(...).Get<JwtConfiguration>()` in that one place. It is cheap, and it beats calling `BuildServiceProvider()`, which spins up a second copy of every singleton you have registered.

Go deeper: the *Binding Configuration and Options Pattern* module covers binding, the options pattern, and the `IOptionsSnapshot` versus `IOptionsMonitor` decision.

4.4 Use ASP.NET Core Identity for users; do not hand-roll password storage

Why it matters

Password storage looks like a small problem and is not. A hand-rolled implementation needs a slow, salted, memory-hard hash; a constant-time comparison so timing does not leak; a safe reset-token generator; and a lockout policy so an attacker cannot try ten million passwords overnight. Getting any one of those wrong is a breach, and you will not find out from your tests.

ASP.NET Core Identity ships all of it: PBKDF2 hashing with per-user salts, a hash format that can be upgraded when the defaults change, token providers for reset and confirmation, and lockout. Use it. The interesting parts of your API are elsewhere.

How to do it right

Extend `IdentityUser` with the fields you actually need, and let your `DbContext` inherit from `IdentityDbContext<TUser>`:

```

public class User : IdentityUser
{
    public string? FirstName { get; set; }
    public string? LastName { get; set; }
}

public class RepositoryContext : IdentityDbContext<User>
{
    public RepositoryContext(DbContextOptions options) : base(options) { }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder); // required, or the Identity tables never appear
        modelBuilder.ApplyConfiguration(new CompanyConfiguration());
        modelBuilder.ApplyConfiguration(new RoleConfiguration());
    }
}

```

Configure the policy in one extension method:

```

public static IServiceCollection ConfigureIdentity(this IServiceCollection services)
{
    services.AddIdentity<User, IdentityRole>(o =>
    {
        o.Password.RequiredLength = 10;
        o.Password.RequireDigit = true;
        o.Password.RequireNonAlphanumeric = false;
        o.User.RequireUniqueEmail = true;

        // Lockout is off for API scenarios unless you ask for it.
        o.Lockout.AllowedForNewUsers = true;
        o.Lockout.MaxFailedAccessAttempts = 5;
        o.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
    })
    .AddEntityFrameworkStores<RepositoryContext>()
    .AddDefaultTokenProviders();

    return services;
}

```

Register users through `UserManager<User>` and never touch the password column yourself:

```

public async Task<IdentityResult> RegisterUser(UserForRegistrationDto dto)
{
    var user = _mapper.Map<User>(dto);
    var result = await _userManager.CreateAsync(user, dto.Password!);

    if (result.Succeeded && dto.Roles is not null)
        await _userManager.AddToRolesAsync(user, dto.Roles);

    return result;
}

```

Length beats complexity. A ten-character minimum with no punctuation requirement is stronger in practice than an eight-character password mangled into `Passw0rd!`. And keep length long but avoid the arbitrary rules that push people towards predictable substitutions.

Lockout deserves a paragraph of its own, because it is off by default. The three lockout options above cost you nothing and turn credential stuffing from a background nuisance into a wall. Configuring them is only half the job, though: lockout counters only increment if you validate the password through a path that records failures. `CheckPasswordAsync` does not. Use `SignInManager` or increment explicitly:

```
public async Task<bool> ValidateUser(UserForAuthenticationDto dto)
{
    _user = await _userManager.FindByNameAsync(dto.UserName!);
    if (_user is null || await _userManager.IsLockedOutAsync(_user))
        return false;

    var valid = await _userManager.CheckPasswordAsync(_user, dto.Password!);

    if (valid)
        await _userManager.ResetAccessFailedCountAsync(_user);
    else
        await _userManager.AccessFailedAsync(_user); // trips lockout at the limit

    return valid;
}
```

One last thing: return the same `401 Unauthorized` for an unknown username, a wrong password, and a locked account. Distinguishing them tells an attacker which usernames exist.

Go deeper: *Implementing Identity in ASP.NET Core Project, Creating Tables and Inserting Roles, and User Creation in the Authentication and Authorization in Web API module.*

4.5 Authorize by role or policy on every non-public endpoint

Why it matters

Authentication proves *who* is calling. It says nothing about what they may do. An API that only authenticates has exactly two access levels: the anonymous public and everyone who has ever registered. The intern's test account can delete the company. The self-service signup form you added last week just handed the world write access to your production data.

The failure mode is also the quietest one in this chapter: the endpoint returns 200 and does the work, so nothing looks broken.

How to do it right

Default to deny. Put `[Authorize]` at the controller level, and open individual actions with `[AllowAnonymous]` where you mean to. That way the mistake is an endpoint that is accidentally locked down, which someone reports in ten minutes, rather than one accidentally left open, which nobody reports at all.

```
[Route("api/companies")]
[ApiController]
[Authorize] // everything here needs a valid token
public class CompaniesController : ControllerBase
{
    private readonly IServiceManager _service;

    public CompaniesController(IServiceManager service) => _service = service;

    [HttpGet(Name = "GetCompanies")]
    public async Task<IActionResult> GetCompanies(CancellationToken ct) =>
        Ok(await _service.CompanyService.GetAllCompaniesAsync(trackChanges: false, ct));

    [HttpDelete("{id:guid}")]
    [Authorize(Roles = "Manager")] // narrower than the controller default
    public async Task<IActionResult> DeleteCompany(Guid id)
    {
        await _service.CompanyService.DeleteCompanyAsync(id, trackChanges: false);
        return NoContent();
    }

    [HttpGet("public-summary")]
    [AllowAnonymous] // deliberate, and visible in review
    public IActionResult Summary() => Ok(new { Message = "Anyone may read this." });
}
```

Role authorization only works if the roles reach the token. Emit them as `ClaimTypes.Role` claims when you build it:

```
private async Task<List<Claim>> GetClaims()
{
    var claims = new List<Claim>
    {
        new(ClaimTypes.Name, _user!.UserName!),
        new(JwtRegisteredClaimNames.Sub, _user.Id),
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    foreach (var role in await _userManager.GetRolesAsync(_user))
        claims.Add(new Claim(ClaimTypes.Role, role));

    return claims;
}
```

The two status codes mean different things, and clients care: **401 Unauthorized** means you did not prove who you are, so authenticate and retry. **403 Forbidden** means we know exactly who you are and the answer is still no, and retrying will not help.

Roles run out of road quickly. Once the rule is "a manager, but only for their own department", move to a policy:

```
services.AddAuthorization(options =>
{
    options.AddPolicy("CanDeleteCompany", policy =>
        policy.RequireRole("Manager")
            .RequireClaim("department", "operations"));
});
```

```
[Authorize(Policy = "CanDeleteCompany")]
```

Policies keep the rule in one place with a name you can search for, instead of scattered across attributes. And whichever you use, write an integration test that calls each protected endpoint with no token, a wrong-role token, and a right-role token. Authorization is the one thing you should never assume still works.

Go deeper: *Protecting Endpoints and Role-Based Authorization*, with the *Authorization Tests* lesson in the *Testing ASP.NET Core Web API* module.

4.6 Issue short-lived access tokens with refresh tokens behind them

Why it matters

A JWT cannot be revoked. Once it is signed and handed out, your API will accept it until the `exp` claim passes. There is no list to remove it from, and no session to kill. So the token's lifetime *is* your breach window. Issue a 24-hour token and a copy pulled out of a log file, a browser extension, or a proxy is good for a full day, even after the user changes their password.

Five-minute tokens fix that, but nobody wants to retype their password every five minutes. Refresh tokens are how you get both.

How to do it right

Issue two credentials: a short-lived access token that carries the claims, and a long-lived refresh token that is a random opaque value stored in your database. The access token expires fast; the refresh token can be revoked because you own the row.

Add the storage to the user:

```
public class User : IdentityUser
{
    public string? FirstName { get; set; }
    public string? LastName { get; set; }
    public string? RefreshToken { get; set; }
    public DateTime RefreshTokenExpiryTime { get; set; }
}
```

Generate the refresh token from a cryptographic RNG, never `Guid.NewGuid()` or `Random`:

```
private static string GenerateRefreshToken()
{
    var bytes = RandomNumberGenerator.GetBytes(32);
    return Convert.ToBase64String(bytes);
}

public async Task<TokenDto> CreateToken(bool populateExp)
{
    var tokenOptions = GenerateTokenOptions(GetSigningCredentials(), await GetClaims());

    _user!.RefreshToken = GenerateRefreshToken();
    if (populateExp)
        _user.RefreshTokenExpiryTime = DateTime.UtcNow.AddDays(7);

    await _userManager.UpdateAsync(_user);

    return new TokenDto(
        new JwtSecurityTokenHandler().WriteToken(tokenOptions),
        _user.RefreshToken);
}
```

Now the part that trips people up. The refresh endpoint receives an access token that has *already expired*. That is the entire reason the client is calling it. So the validation you run when reading claims out of that token must skip the lifetime check:

```
private ClaimsPrincipal GetPrincipalFromExpiredToken(string token)
{
    var parameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = _jwt.ValidIssuer,
        ValidAudience = _jwt.ValidAudience,
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(Environment.GetEnvironmentVariable("SECRET")!)),

        // Deliberate: this endpoint exists to accept expired access tokens.
        ValidateLifetime = false
    };

    var principal = new JwtSecurityTokenHandler()
        .ValidateToken(token, parameters, out var securityToken);

    if (securityToken is not JwtSecurityToken jwt ||
```

```

        !jwt.Header.Alg.Equals(SecurityAlgorithms.HmacSha256,
            StringComparison.InvariantCultureIgnoreCase))
        throw new SecurityTokenException("Invalid token");

    return principal;
}

```

This is not a hole. Issuer, audience, signature, and algorithm are all still checked, so the token is provably yours. You are only declining to reject it for being old. The real gate is the refresh token in the database, checked next:

```

public async Task<TokenDto> RefreshToken(TokenDto tokenDto)
{
    var principal = GetPrincipalFromExpiredToken(tokenDto.AccessToken);

    var user = await _userManager.FindByNameAsync(principal.Identity!.Name!);
    if (user is null ||
        user.RefreshToken != tokenDto.RefreshToken ||
        user.RefreshTokenExpiryTime <= DateTime.UtcNow)
        throw new RefreshTokenBadRequest();

    _user = user;
    return await CreateToken(populateExp: false); // rotate, keep the original expiry
}

```

The course takes the stricter line and leaves `ValidateLifetime = true`, which forces clients to refresh *before* expiry, a defensible choice if you also control the client, and the lesson says so explicitly. If you are shipping an API other people call, `false` is the practical default.

Three things to hold on to. **Rotate on every refresh:** the code above overwrites `RefreshToken`, so the old one dies. **Do not extend the expiry on refresh** (`populateExp: false`), or the session becomes immortal. And an unexpected refresh token means the user's session was replayed: clear the column and force a login.

Go deeper: the *Refresh Token In ASP.NET Core Web API* module: *The Need for Refresh Tokens*, *Refresh Token Implementation*, and *Token Controller Implementation*.

4.7 Configure CORS deliberately, not `AllowAnyOrigin`

Why it matters

CORS is the browser asking your API a question: *may this page, served from this origin, read your response?* `AllowAnyOrigin()` answers "yes, anybody", which is fine for a genuinely public read-only API and wrong for everything else. It means any site on the internet can serve JavaScript that calls your API from a visitor's browser and reads the results.

It is also a trap that fails at the worst moment. `AllowAnyOrigin()` and `AllowCredentials()` are illegal together, and ASP.NET Core throws at startup if you combine them. So the day you move to cookie auth, the wide-open policy you copied from a tutorial stops the application from booting, and the fastest-looking fix is usually the wrong one.

How to do it right

Name the origins. Name the methods. Name the headers. Read them from configuration so each environment gets its own list.

```
"Cors": {
  "AllowedOrigins": [ "https://app.example.com", "https://admin.example.com" ]
}
```

```
public static IServiceCollection ConfigureCors(
    this IServiceCollection services, IConfiguration configuration)
{
    var allowedOrigins = configuration
        .GetSection("Cors:AllowedOrigins").Get<string[]>() ?? [];

    services.AddCors(options =>
    {
        options.AddPolicy("CorsPolicy", policy =>
            policy.WithOrigins(allowedOrigins)
                .WithMethods("GET", "POST", "PUT", "PATCH", "DELETE")
                .WithHeaders("Authorization", "Content-Type", "Accept")
                .WithExposedHeaders("X-Pagination")
                .AllowCredentials()
                .SetPreflightMaxAge(TimeSpan.FromMinutes(10)));
    });

    return services;
}
```

```
app.UseCors("CorsPolicy");
```

`WithExposedHeaders("X-Pagination")` is the line everybody forgets, and it is the one that makes your paging work. By default a browser only lets JavaScript read a handful of "safe-listed" response headers. Your carefully built pagination metadata from item 5.1 arrives on the wire, sits in the response, and `response.headers.get('X-Pagination')` returns `null`, for a cross-origin caller only. It works perfectly in Postman, which is why this bug survives so long. Any custom response header your clients need to read must be listed here.

The rest of the policy, briefly:

- `WithOrigins` takes exact scheme-host-port matches. `https://app.example.com` does not cover `http://app.example.com` or `https://app.example.com:8443`. There is no wildcard subdomain support without `SetIsOriginAllowed`, and if you write that predicate, write it carefully, because a naive `EndsWith("example.com")` matches `evilexample.com`.

- `AllowCredentials` is only needed for cookies or TLS client certificates. Bearer tokens in an `Authorization` header do not need it. You only need `Authorization` in `WithHeaders`. Leave it off unless you use cookies.
- `SetPreflightMaxAge` lets the browser cache the `OPTIONS` preflight, which removes a round trip from every non-simple request.

Two things CORS is not. It is not authorization. It restricts browsers, and a browser is the only thing that honours it. `curl`, a mobile app, and a server-side script ignore CORS entirely, so `[Authorize]` from item 4.5 is still doing all the real work. And its placement in the pipeline matters: `UseCors` goes before `UseResponseCaching` and after `UseRateLimiter`, or you will cache a response with the wrong CORS headers on it.

Go deeper: *Extension Methods and CORS Configuration* in the *Project Configuration* module.

What this chapter had to leave out

- Wiring ASP.NET Core Identity into a real user store (the Identity tables, role seeding, user creation, and login) instead of the configuration sketch this chapter had room for.
- The complete refresh-token flow: why they exist, the implementation, the token controller both your clients talk to, and token rotation across a real request cycle.
- Why `ValidateLifetime` is `false` on the expired-token path, what that costs you, and what has to be true elsewhere for it to stay safe.
- Role-based authorization proved rather than assumed, in an authorization test class that demonstrates a locked endpoint really is locked.
- `IOptionsSnapshot` versus `IOptionsMonitor`, and which one your JWT options actually need once the signing secret can change without a restart.
- **Mastering ASP.NET Core Security**, a whole second course, free with your purchase, for everything past the token.

Security is the half of an API you cannot afford to half-learn. The course builds all of it, and throws in a second security course free. [Get Ultimate ASP.NET Core Web API](#)

Chapter 5: Performance & Scale

Most APIs are fast in development because development has thirty rows and one user. Production has three hundred thousand rows, a mobile client with a retry loop, and a crawler that discovered your list endpoint. The six items in this chapter are the ones that decide whether your API degrades gracefully under that load or falls over: bounded result sets, non-blocking queries, server-side filtering, safe dynamic queries, caching you can invalidate, and a rate limiter that says no politely.

5.1 Never return an unbounded collection; page everything

Why it matters

`GET /api/companies/{id}/employees` returning every row is a bug waiting for data. It passes every test you write, because your seed data has eight employees. The day a customer imports 300,000 rows, that single endpoint materialises the whole table into memory, serialises it into a response body measured in megabytes, and holds a connection open long enough to starve everything else. The client is no better off. It has limited memory too, and it only ever renders twenty rows.

The fix is not "add a page size parameter". The fix is that the *unpaged* request must also be bounded. If a caller sends no parameters at all, they still get one page.

How to do it right

Put the paging parameters in a base class that every entity's parameter object inherits, and clamp the page size in the setter so a caller cannot ask for a million rows:

```
public abstract class RequestParameters
{
    private const int MaxPageSize = 50;
    private int _pageSize = 10;

    public int PageNumber { get; set; } = 1;

    public int PageSize
    {
        get => _pageSize;
        set => _pageSize = value > MaxPageSize ? MaxPageSize : value;
    }
}

public class EmployeeParameters : RequestParameters;
```

Defaults matter as much as the clamp: page 1, size 10, both applied when the query string is empty.

Push `Skip / Take` into the query, and count separately. Counting a filtered `IQueryable` on the server is one extra round trip; materialising the table to call `.Count()` in memory is a disaster:

```
public async Task<PagedList<Employee>> GetEmployeesAsync(Guid companyId,
    EmployeeParameters parameters, bool trackChanges, CancellationToken ct = default)
{
    var query = FindByCondition(e => e.CompanyId.Equals(companyId), trackChanges)
        .OrderBy(e => e.Name);

    var count = await query.CountAsync(ct);

    var employees = await query
        .Skip((parameters.PageNumber - 1) * parameters.PageSize)
        .Take(parameters.PageSize)
        .ToListAsync(ct);

    return new PagedList<Employee>(employees, count, parameters.PageNumber,
        parameters.PageSize);
}
```

`PagedList<T>` carries the page itself plus the metadata a client needs to build a pager:

```
public class MetaData
{
    public int CurrentPage { get; init; }
    public int TotalPages { get; init; }
    public int PageSize { get; init; }
    public int TotalCount { get; init; }

    public bool HasPrevious => CurrentPage > 1;
    public bool HasNext => CurrentPage < TotalPages;
}

public class PagedList<T>(List<T> items, int count, int pageNumber, int pageSize) : List<T>
(items)
{
    public MetaData MetaData { get; } = new()
    {
        TotalCount = count,
        PageSize = pageSize,
        CurrentPage = pageNumber,
        TotalPages = (int)Math.Ceiling(count / (double)pageSize)
    };
}
```

The metadata does not belong in the body, because the body is a collection of employees, not a wrapper object. Put it in a response header:

```
[HttpGet]
public async Task<IActionResult> GetEmployeesForCompany(Guid companyId,
    [FromQuery] EmployeeParameters parameters, CancellationToken ct)
{
    var (employees, metaData) = await _service.EmployeeService
        .GetEmployeesAsync(companyId, parameters, trackChanges: false, ct);

    Response.Headers.Append("X-Pagination", JsonSerializer.Serialize(metaData));

    return Ok(employees);
}
```

Use `Append`, not `Add`. `Add` throws an `ArgumentException` if the key is already present.

One trap that bites in production and never in Postman: a browser client on another origin cannot read `X-Pagination` unless CORS exposes it. Your paging header is invisible to the very frontend it was built for until you add `.WithExposedHeaders("X-Pagination")` to your CORS policy (see item 4.7).

Go deeper: *Paging in ASP.NET Core Web API*, where the "Improving the Solution" and "Additional Improvement" lessons walk through the `PagedList` refactor and the count-before-skip optimisation measured on millions of rows.

5.2 Make queries async and read-only by default

Why it matters

Two habits decide how many concurrent requests one instance can serve. The first is blocking: calling `.Result` or `.Wait()` on a task parks a thread-pool thread doing nothing while the database works. Under load, the pool runs dry, latency climbs on requests that have nothing to do with the slow query, and the whole app looks broken for reasons that never appear in a single-user test.

The second is change tracking. EF Core builds a tracking snapshot of every entity it returns so it can detect modifications on save. On a read endpoint you never save anything, so every one of those snapshots is memory and CPU spent for nothing.

How to do it right

Make the whole chain (repository, service, controller) `async` and return `Task<T>`. Never bridge back to sync:

```
// Don't. This blocks a thread and can deadlock.
public async Task<IEnumerable<Company>> GetAllCompaniesAsync(bool trackChanges) =>
    FindAll(trackChanges).OrderBy(c => c.Name).ToListAsync().Result;
```

Flow a `CancellationToken` from the action all the way to EF Core. ASP.NET Core cancels that token when the client disconnects, so an abandoned request stops costing you a database query:

```
[HttpGet]
public async Task<IActionResult> GetCompanies(CancellationToken ct)
{
    var companies = await _service.CompanyService.GetAllCompaniesAsync(trackChanges: false, ct);
    return Ok(companies);
}
```

```
public async Task<IEnumerable<Company>> GetAllCompaniesAsync(bool trackChanges,
    CancellationToken ct = default) =>
    await FindAll(trackChanges)
        .OrderBy(c => c.Name)
        .ToListAsync(ct);
```

The token also belongs on writes, so pass it to `SaveChangesAsync(ct)` in your unit-of-work `SaveAsync`. Note that a *synchronous* repository method like `Create(entity)` does not need one; it only stages the entity in the change tracker.

Make tracking an explicit decision rather than an accident. A `trackChanges` flag on the base repository puts `AsNoTracking()` on every read path:

```
public abstract class RepositoryBase<T>(RepositoryContext context) : IRepositoryBase<T>
    where T : class
{
    protected RepositoryContext RepositoryContext = context;

    public IQueryable<T> FindAll(bool trackChanges) =>
        !trackChanges
            ? RepositoryContext.Set<T>().AsNoTracking()
            : RepositoryContext.Set<T>();

    public IQueryable<T> FindByCondition(Expression<Func<T, bool>> expression, bool
trackChanges) =>
        !trackChanges
            ? RepositoryContext.Set<T>().Where(expression).AsNoTracking()
            : RepositoryContext.Set<T>().Where(expression);
}
```

Then every call site reads as a statement of intent: `trackChanges: false` for a GET, `trackChanges: true` only when the entity you fetched is about to be modified and saved.

One more caution on cancellation: a cancelled request throws `OperationCanceledException` out of EF Core. That is expected behaviour, not a fault, so make sure your global exception handler (item 3.1) does not log every client disconnect as an error.

Go deeper: *Asynchronous Code*, with the "Common Pitfalls" lesson on `Result` / `Wait()` deadlocks and the "Cancellation Token" lesson that threads the token from action to `ToListAsync`.

5.3 Filter, search, and sort on the server, not in the client

Why it matters

The single most common cause of "why is my API slow" is a query that got materialised too early. Someone calls `.ToList()` or `.ToListAsync()`, and then filters. From that line onward you are no longer talking to the database. You are doing LINQ-to-objects over a fully loaded table. The generated SQL says `SELECT * FROM Employees`, the filter runs in your process, and the endpoint gets slower in exact proportion to how much data your customers have.

The same rule applies to sorting. Sorting after materialisation means you loaded rows you were about to discard on the next page.

How to do it right

Keep everything an `IQueryable` and compose. Write each concern as an extension method that takes and returns `IQueryable<T>`, so nothing executes until the final `ToListAsync`:

```
public static class RepositoryEmployeeExtensions
{
    public static IQueryable<Employee> FilterEmployees(this IQueryable<Employee> employees,
        uint minAge, uint maxAge) =>
        employees.Where(e => e.Age >= minAge && e.Age <= maxAge);

    public static IQueryable<Employee> Search(this IQueryable<Employee> employees, string?
searchTerm)
    {
        if (string.IsNullOrEmpty(searchTerm))
            return employees;

        var lowerCaseTerm = searchTerm.Trim().ToLower();

        return employees.Where(e => e.Name.ToLower().Contains(lowerCaseTerm));
    }
}
```

Note the early return in `Search`. A no-op filter must return the query untouched rather than adding a `WHERE` clause that matches everything.

Now the repository method is filter, search, sort, count, page: one SQL statement for the count and one for the page:

```

public async Task<PagedList<Employee>> GetEmployeesAsync(Guid companyId,
    EmployeeParameters parameters, bool trackChanges, CancellationToken ct = default)
{
    var query = FindByCondition(e => e.CompanyId.Equals(companyId), trackChanges)
        .FilterEmployees(parameters.MinAge, parameters.MaxAge)
        .Search(parameters.SearchTerm)
        .Sort(parameters.OrderBy);

    var count = await query.CountAsync(ct);

    var employees = await query
        .Skip((parameters.PageNumber - 1) * parameters.PageSize)
        .Take(parameters.PageSize)
        .ToListAsync(ct);

    return new PagedList<Employee>(employees, count, parameters.PageNumber,
        parameters.PageSize);
}

```

The parameters object grows alongside it, and it is the right place for cheap input rules:

```

public class EmployeeParameters : RequestParameters
{
    public EmployeeParameters() => OrderBy = "name";

    public uint MinAge { get; set; }
    public uint MaxAge { get; set; } = int.MaxValue;
    public string? SearchTerm { get; set; }

    public bool ValidAgeRange => MaxAge > MinAge;
}

```

`uint` makes a negative age unrepresentable rather than something you validate for. `MaxAge` defaults high so a caller can supply only `minAge`. And `ValidAgeRange` catches the nonsense request (`minAge=40&maxAge=20`) before it reaches the database, throwing a typed exception your handler maps to 400 (item 3.3).

Two production caveats the composition style makes easy to forget. `e.Name.ToLower().Contains(...)` prevents index usage on most providers. For anything past a demo, rely on a case-insensitive collation or a real full-text index instead. And every column a caller can filter or sort on is a column that needs an index; an unindexed sort on a large table is a full scan no matter how clean the LINQ looks.

Go deeper: the *Filtering*, *Searching*, and *Sorting* chapters build these extension methods one at a time and show the generated queries at each step.

5.4 Validate anything you interpolate into a dynamic query

Why it matters

`?orderBy=name,age desc` is a caller-supplied string that ends up inside a query. If you concatenate it straight into dynamic LINQ or, worse, into raw SQL, you have handed a stranger a piece of your query planner. At best they order by a column you never meant to expose and learn something about your schema; at worst, with raw SQL, it is textbook injection wearing a friendly parameter name.

Parameterisation does not save you here, which is what makes this trap different from the usual SQL-injection advice. You can parameterise *values*. You cannot parameterise an *identifier*, because a column name or a sort direction has to be part of the query text. So identifiers need a whitelist.

How to do it right

Build the order clause by reflecting over the target type and keeping only names that actually exist as public instance properties. Anything unrecognised is dropped, not passed through:

```
public static class OrderQueryBuilder
{
    public static string CreateOrderQuery<T>(string orderByQueryString)
    {
        var orderParams = orderByQueryString.Trim().Split(',');
        var propertyInfos = typeof(T).GetProperties(BindingFlags.Public |
BindingFlags.Instance);
        var orderQueryBuilder = new StringBuilder();

        foreach (var param in orderParams)
        {
            if (string.IsNullOrEmpty(param))
                continue;

            var propertyFromQueryName = param.Split(' ')[0];
            var objectProperty = propertyInfos.FirstOrDefault(pi =>
                pi.Name.Equals(propertyFromQueryName,
                    StringComparison.InvariantCultureIgnoreCase));

            if (objectProperty is null)
                continue;

            var direction = param.EndsWith(" desc") ? "descending" : "ascending";

            orderQueryBuilder.Append($"{objectProperty.Name} {direction}, ");
        }

        return orderQueryBuilder.ToString().TrimEnd(',', ' ');
    }
}
```

The important detail is that the string appended is `objectProperty.Name`, the name as declared on your type, never the caller's text. The caller's input only ever selects which known property to use. Direction is likewise reduced to one of two constants.

Then fall back to a safe default whenever the whitelist leaves you with nothing:

```
public static IQueryable<Employee> Sort(this IQueryable<Employee> employees, string?
orderByQueryString)
{
    if (string.IsNullOrEmpty(orderByQueryString))
        return employees.OrderBy(e => e.Name);

    var orderQuery = OrderQueryBuilder.CreateOrderQuery<Employee>(orderByQueryString);

    if (string.IsNullOrEmpty(orderQuery))
        return employees.OrderBy(e => e.Name);

    return employees.OrderBy(orderQuery); // System.Linq.Dynamic.Core
}
```

Never return an unordered query from a paged endpoint. Without a deterministic sort, `Skip` / `Take` can return the same row on two different pages and skip another entirely.

Generalise the rule beyond sorting. Anywhere a caller-supplied string becomes part of query *structure* (a column in a dynamic filter, a table name, a Dapper `ORDER BY`, an `IN` list built by hand), the same discipline applies: map the input to a value from a set you control, and never let the raw string through. Values go in parameters; identifiers come from a whitelist. If you want to narrow the surface further, whitelist an explicit set of sortable property names instead of every public property on the entity, because reflection over the type also exposes columns you may not want callers ordering by.

Go deeper: *Sorting in ASP.NET Core Web API*, where "Improving the Sorting Implementation" extracts this into the generic `OrderQueryBuilder.CreateOrderQuery<T>` so every entity gets the same protection.

5.5 Cache responses that can tolerate being slightly stale, and know what invalidates them

Why it matters

A reference endpoint hit thousands of times a minute should not hit the database thousands of times a minute. Caching is the cheapest performance win available to you, and it is also the easiest way to ship a bug you cannot see: the response is correct, the status code is 200, and the data is forty seconds out of date. Nobody notices until a customer updates a record, refreshes, and sees the old value staring back.

So the caching question is never "how long do I cache this". It is "what makes this entry wrong, and what removes it when that happens".

How to do it right

There are three layers, and they are not alternatives. They solve different problems.

Output caching is your default. The server decides what is cached and for how long, and the client cannot bypass it with a `no-cache` header. It supports resource locking (which prevents a cache stampede when a hot entry expires), tagging, and eviction. Register it, add the middleware after CORS, and define named policies rather than scattering durations across attributes:

```
builder.Services.AddOutputCache(opt =>
{
    opt.AddBasePolicy(bp => bp.Expire(TimeSpan.FromSeconds(10)));

    opt.AddPolicy("Companies", p => p
        .Expire(TimeSpan.FromSeconds(120))
        .Tag("companies"));
});
```

```
app.UseCors("CorsPolicy");
app.UseOutputCache();
```

```
[HttpGet(Name = "GetCompanies")]
[OutputCache(PolicyName = "Companies")]
public async Task<IActionResult> GetCompanies(CancellationToken ct) { /* ... */ }
```

An attribute on an action overrides the base policy. By default only 200 responses to GET/HEAD are cached, cookie-setting and authenticated responses are not.

Evict on write. This is the part most tutorials leave out, and it is the whole difference between a cache and a bug. A tagged policy gives you a handle; inject `IOutputCacheStore` and drop the tag whenever the underlying data changes:

```
[HttpPut("{id:guid}")]
public async Task<IActionResult> UpdateCompany(Guid id,
    [FromBody] CompanyForUpdatedDto company,
    [FromServices] IOutputCacheStore cacheStore,
    CancellationToken ct)
{
    await _service.CompanyService.UpdateCompanyAsync(id, company, trackChanges: true, ct);

    await cacheStore.EvictByTagAsync("companies", ct);

    return NoContent();
}
```

Without that call, a PUT followed by a GET inside the cache window returns the old body: correct-looking, and wrong.

ETags and 304 revalidation save bandwidth, not database work. The client sends back the last ETag in `If-None-Match`; if it still matches, you answer `304 Not Modified` with no body. The ETag must therefore be derived from the resource's actual state (a hash or a rowversion) so it changes exactly when the resource does. An ETag generated fresh per request (`Guid.NewGuid()`) will never match and buys you nothing:

```
[HttpGet("{id:guid}", Name = "CompanyById")]
public async Task<IActionResult> GetCompany(Guid id, CancellationToken ct)
{
    var company = await _service.CompanyService.GetCompanyAsync(id, trackChanges: false, ct);

    var etag = $"\"{Convert.ToHexString(
        SHA256.HashData(JsonSerializer.SerializeToUtf8Bytes(company)))[..32]}\"";

    if (Request.Headers.IfNoneMatch.Contains(etag))
        return StatusCode(StatusCodes.Status304NotModified);

    Response.Headers.ETag = etag;

    return Ok(company);
}
```

A stored `RowVersion` / `ConcurrencyToken` column is a cheaper source than hashing the payload, so use it where you have one.

Response caching is the older attribute you will meet in existing codebases:

`[ResponseCache(Duration = 60)]` plus `AddResponseCaching()`, with reusable `CacheProfiles` registered in `AddControllers`. It works through standard HTTP cache headers, which means the client can opt out with `no-cache` and you have no eviction story at all. Understand it so you can read old code; reach for output caching in new code.

Go deeper: *Caching in ASP.NET Core*, from "Adding Cache Headers" through "Output Caching", "Output Cache Keys", and "Caching Revalidation", which demonstrates the stale-after-PUT behaviour live.

5.6 Rate limit before someone finds out you did not

Why it matters

You do not need an attacker to be taken down. One mobile client with a retry loop and no backoff, one misconfigured cron job, one integration partner who loops instead of paging: any of these will hammer an unprotected endpoint until your database connection pool is gone. Rate limiting is the difference between "some callers get 429s for a minute" and "the API is down for everyone".

ASP.NET Core has this built in. There is no package to evaluate and no reverse proxy required.

How to do it right

Configure a global limiter and register the middleware before CORS:

```
public static void ConfigureRateLimitingOptions(this IServiceCollection services) =>
    services.AddRateLimiter(opt =>
    {
        opt.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(context =>
            RateLimitPartition.GetFixedWindowLimiter("GlobalLimiter",
                _ => new FixedWindowRateLimiterOptions
                {
                    AutoReplenishment = true,
                    PermitLimit = 100,
                    QueueLimit = 2,
                    QueueProcessingOrder = QueueProcessingOrder.OldestFirst,
                    Window = TimeSpan.FromMinutes(1)
                }
            ));

        opt.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    });
```

```
app.UseRateLimiter();
app.UseCors("CorsPolicy");
```

Two things here are easy to get wrong.

First, **the default rejection status is 503**, not 429. That is a lie to your callers: 503 means "the server is broken, back off from everything", while 429 means "you specifically sent too many, here is when to retry". Set `RejectionStatusCode` explicitly, always.

Second, a small `QueueLimit` **beats a hard reject for bursty traffic**. With `QueueLimit = 2`, two excess requests wait for the next window instead of failing. Keep it small, because a long queue turns a throughput problem into a latency problem.

Then make the rejection response useful. Emit a `Retry-After` header, which well-behaved clients and HTTP libraries already honour, and a problem-details body so it parses like every other error your API returns (item 3.2):

```
opt.OnRejected = async (context, token) =>
{
    context.HttpContext.Response.StatusCode = StatusCodes.Status429TooManyRequests;

    var retryAfterSeconds = context.Lease.TryGetMetadata(MetadataName.RetryAfter, out var
retryAfter)
        ? (int)retryAfter.TotalSeconds
        : 60;

    context.HttpContext.Response.Headers.RetryAfter = retryAfterSeconds.ToString();

    await context.HttpContext.Response.WriteAsJsonAsync(new ProblemDetails
    {
        Status = StatusCodes.Status429TooManyRequests,
        Type = "https://tools.ietf.org/html/rfc6585#section-4",
        Title = "Too many requests",
        Detail = $"Rate limit exceeded. Try again in {retryAfterSeconds} second(s).",
    }, options: null, contentType: "application/problem+json", cancellation token: token);
};
```

Named policies let you tighten specific endpoints (login, search, anything expensive) without changing the global rule:

```
opt.AddPolicy("Login", context =>
    RateLimitPartition.GetFixedWindowLimiter("LoginLimiter",
        _ => new FixedWindowRateLimiterOptions
        {
            AutoReplenishment = true,
            PermitLimit = 5,
            Window = TimeSpan.FromMinutes(1)
        }));
```

```
[HttpPost("login")]
[EnableRateLimiting("Login")]
public async Task<IActionResult> Authenticate([FromBody] UserForAuthenticationDto user) { /* ...
*/ }
```

A named policy does not replace the global limiter. Both apply. Use `[DisableRateLimiting]` to exempt an endpoint entirely; your health check endpoint (item 6.2) is the usual candidate, since you do not want a load balancer's probes counted against a limit.

One caveat before you ship: the built-in limiter keeps its counters **in process**. Across three instances behind a load balancer, a `PermitLimit` of 100 is effectively 300. Partition by something meaningful (API key, authenticated user id) rather than a single constant, size the limit with the instance count in mind, and for a hard global cap use a distributed limiter or enforce it at the gateway.

Go deeper: *Rate Limiting and Throttling*, where "Implementing Rate Limiting" covers the fixed-window limiter, the surprising 503 default, queueing, and per-endpoint policies.

What this chapter had to leave out

- The paging refactor in full: the `PagedList`, the concrete query, and the count-before-skip optimisation timed against millions of rows, so "materialise late" becomes something you have watched rather than something you were told.
- Filtering, searching, and sorting built one module at a time, with the generated SQL shown at every step.
- The generic `OrderQueryBuilder.CreateOrderQuery<T>` that gives every entity in the project the same whitelist protection without you writing it twice.
- Seven caching lessons: cache headers, the cache store, output caching, output cache keys, and revalidation demonstrated live against a response that has gone stale after a PUT.
- The async pitfalls that only bite in production: `.Result` and `.Wait()` deadlocks, and threading a `CancellationToken` from the action all the way down to `ToListAsync`.

Every claim in this chapter is measured on real data in the course. [Get Ultimate ASP.NET Core Web API](#)

Chapter 6: Observability & Documentation

An API you cannot see into is an API you cannot operate. Once your code is on a server you no longer have a debugger, a watch window, or a stack of breakpoints. You have logs, health endpoints, and whatever documentation you shipped. These three items are what turn "it's broken and I don't know why" into "it's the database, and here is the request that proves it".

6.1 Log through a structured logger, and never log secrets

Why it matters

At 3am the only thing standing between you and a guess is your log. `Console.WriteLine($"User {id} failed")` produces a line of text nobody can query: you cannot ask "show me every failure for this user in the last hour" because the user id is baked into a string, not stored as a field. Worse is the other direction: teams that log the whole request body to "be safe" and end up with passwords, bearer tokens, and card numbers sitting in plain text in a log aggregator half the company can read. A log that cannot be searched is useless; a log that leaks credentials is a liability.

How to do it right

Use a real logging library with structured sinks. Serilog is the usual pick: configure it from `appsettings.json` so you can change sinks and levels without a rebuild, and route the framework's own logs through the same pipeline.

```
{
  "Serilog": {
    "Using": [ "Serilog.Sinks.Console", "Serilog.Sinks.File", "Serilog.Sinks.Seq" ],
    "MinimumLevel": {
      "Default": "Information",
      "Override": { "Microsoft.AspNetCore": "Warning" }
    },
    "WriteTo": [
      { "Name": "Console" },
      {
        "Name": "File",
        "Args": { "Path": "logs/log-.txt", "RollingInterval": "Day" }
      },
      {
        "Name": "Seq",
        "Args": { "serverUrl": "http://localhost:5341" }
      }
    ],
    "Enrich": [ "FromLogContext" ]
  }
}
```

```
using Serilog;

var builder = WebApplication.CreateBuilder(args);

builder.Host.UseSerilog((context, configuration) =>
    configuration.ReadFrom.Configuration(context.Configuration));
```

The rule that makes the whole thing pay off is **message templates, not string interpolation**. The two lines below print the same text, but only the first stores `CompanyId` as a queryable property:

```
// Good: structured. Seq/Elastic/Application Insights can filter on CompanyId.
logger.LogWarning("Company {CompanyId} was not found for tenant {TenantId}", id, tenantId);

// Bad: one opaque string. Nothing to filter on.
logger.LogWarning($"Company {id} was not found for tenant {tenantId}");
```

The second rule: when you log an exception, log the **exception object**, not `exception.Message`. `Message` throws away the stack trace and the inner exception, exactly the two things you needed.

```
public sealed class GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger)
    : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext,
        Exception exception, CancellationToken cancellationToken)
    {
        // exception is the first argument, not string-formatted into the message.
        logger.LogError(exception,
            "Unhandled exception on {Method} {Path} (trace {TraceId})",
            httpContext.Request.Method, httpContext.Request.Path,
            httpContext.TraceIdentifier);

        return true; // problem details written elsewhere, see item 3.2
    }
}
```

Correlate requests so one failure can be followed end to end. `HttpContext.TraceIdentifier` is free and already flows into `ProblemDetails`; push it into the log context once, in middleware, and every log line inside that request carries it:

```
app.Use(async (context, next) =>
{
    using (Serilog.Context.LogContext.PushProperty("TraceId", context.TraceIdentifier))
    {
        await next();
    }
});
```

Finally, keep an explicit never-log list and treat it as a code review item: passwords and password-reset tokens, access and refresh tokens, `Authorization` and `Cookie` headers, connection strings, API keys, full request and response bodies on auth endpoints, and personal data you have no reason to keep. Log

identifiers instead of values: `UserId`, not the email; `PaymentId`, not the card. If you need the body for debugging, log it only in Development and only behind a flag.

Go deeper: the course builds this logging service from scratch. See *Configuring a Logging Service*, and in particular the *Adding a New Sink - Seq* lesson, where the structured half of Serilog finally shows what it is for.

6.2 Expose a health check endpoint your orchestrator can actually use

Why it matters

A process can be running and still be useless. The database connection died, the pool is exhausted, the third-party service you depend on is timing out, but the app still accepts TCP connections, so your load balancer keeps sending it traffic and your users keep getting 500s. A health endpoint is how the platform finds out the difference between "the process is alive" and "this instance can serve requests", and it is the difference between a ten-second blip and a ten-minute outage.

How to do it right

Register the checks you actually depend on, tag them, and expose more than one endpoint. Start with the packages:

```
dotnet add package AspNetCore.HealthChecks.UI.Client
dotnet add package AspNetCore.HealthChecks.SqlServer
dotnet add package AspNetCore.HealthChecks.UI
dotnet add package AspNetCore.HealthChecks.UI.InMemory.Storage
```

Then wire the checks up in an extension method so `Program.cs` stays readable:

```
public static void ConfigureHealthChecks(this IServiceCollection services,
    IConfiguration configuration)
{
    services.AddHealthChecks()
        .AddSqlServer(configuration.GetConnectionString("sqlConnection"),
            name: "Sql Health", tags: ["database", "ready"])
        .AddCheck<PaymentGatewayHealthCheck>("PaymentGateway", tags: ["ready"]);

    services.AddHealthChecksUI()
        .AddInMemoryStorage();
}
```

A custom check is one interface. Return `Degraded` when you are slow but working, `Unhealthy` when you are not:

```

public sealed class PaymentGatewayHealthCheck(IHttpClientFactory factory) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        try
        {
            using var client = factory.CreateClient("payments");
            var started = Stopwatch.GetTimestamp();
            var response = await client.GetAsync("/ping", cancellationToken);
            var elapsed = Stopwatch.GetElapsedTime(started);

            if (!response.IsSuccessStatusCode)
                return HealthCheckResult.Unhealthy($"Gateway returned
{(int)response.StatusCode}.");

            return elapsed > TimeSpan.FromSeconds(2)
                ? HealthCheckResult.Degraded($"Gateway slow: {elapsed.TotalMilliseconds:N0}
ms.")
                : HealthCheckResult.Healthy();
        }
        catch (Exception ex)
        {
            return HealthCheckResult.Unhealthy("Gateway unreachable.", ex);
        }
    }
}

```

Now the part your platform team will ask for: **liveness and readiness are different questions.**

- **Liveness:** "is this process wedged and in need of a restart?" It must check nothing external. If you make liveness depend on the database, a database blip restarts every instance you own at once.
- **Readiness:** "should this instance receive traffic right now?" This one checks dependencies, and a failure removes the instance from the rotation instead of killing it.

```

public static void ConfigureHealthChecksEndpoints(this WebApplication app)
{
    // Liveness: no dependencies at all. Only fails if the process is broken.
    app.MapHealthChecks("/health/live", new HealthCheckOptions
    {
        Predicate = _ => false
    });

    // Readiness: everything this instance needs to serve a request.
    app.MapHealthChecks("/health/ready", new HealthCheckOptions
    {
        Predicate = reg => reg.Tags.Contains("ready"),
        ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
    });

    app.MapHealthChecksUI();
}

```

Point the container's liveness probe (or the load balancer's) at `/health/live` and the readiness probe at `/health/ready`. Two more details worth getting right: the overall status is the worst individual status, so one `Degraded` check degrades the whole report. Tag carefully. And a health endpoint that dumps exception details is reconnaissance for an attacker: keep the detailed writer and the UI dashboard behind authentication or an internal-only route, and leave `/health/live` as a bare status code.

The UI is optional but cheap. Add the polling config and you get a dashboard at `/healthchecks-ui`:

```
"HealthChecksUI": {
  "HealthChecks": [ { "Name": "API", "Uri": "/health/ready" } ],
  "EvaluationTimeInSeconds": 5
}
```

Go deeper: the course's *Health Checks in ASP.NET Core Web API* chapter walks the whole set: the basic probe, the SQL Server check, a custom `IHealthCheck`, tag filtering, and the health checks UI.

6.3 Ship real OpenAPI docs, with auth and response types wired in

Why it matters

Documentation is not a nicety on an API. It is the interface. A Swagger page with no `Authorize` button means every consumer's first experience is a 401 they cannot get past. A page that lists only `200 OK` under every action means the client developer discovers your 422 in production, from a bug report. Both cost you the same thing: support time you spend explaining what the page should have said.

How to do it right

Swashbuckle gets you the generated document and the UI. Give it a real `OpenApiInfo` per version, and register a security definition so the UI can send a token:

```
public static void ConfigureSwagger(this IServiceCollection services)
{
    services.AddSwaggerGen(s =>
    {
        s.SwaggerDoc("v1", new OpenApiInfo
        {
            Title = "Company Employees API",
            Version = "v1",
            Description = "Companies and their employees.",
            Contact = new OpenApiContact { Name = "API team", Email = "api@example.com" },
            License = new OpenApiLicense { Name = "MIT", Url = new
Uri("https://example.com/license") }
        });
        s.SwaggerDoc("v2", new OpenApiInfo { Title = "Company Employees API", Version = "v2" });

        s.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
        {
```

```

        In = ParameterLocation.Header,
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        Description = "Paste the JWT here. The 'Bearer ' prefix is added for you."
    });

    s.AddSecurityRequirement(document => new OpenApiSecurityRequirement
    {
        [new OpenApiSecuritySchemeReference("Bearer", document)] = []
    });

    var xmlFile = $"
{typeof(Infrastructure.Presentation.AssemblyReference).Assembly.GetName().Name}.xml";
    s.IncludeXmlComments(Path.Combine(AppContext.BaseDirectory, xmlFile));
    });
}

```

XML comments only appear if the project that holds your controllers generates them, so turn the documentation file on there, and suppress CS1591 unless you genuinely intend to comment every public member:

```

<PropertyGroup>
  <GenerateDocumentationFile>true</GenerateDocumentationFile>
  <NoWarn>$(NoWarn);1591</NoWarn>
</PropertyGroup>

```

Now the part that turns a listing into a contract. Every status code you promised in item 2.4 should appear in the document, with the type of the body attached. The generic form of `[ProducesResponseType]` carries the payload type as well as the code, so consumers can generate a client from your spec instead of guessing:

```

/// <summary>Creates a company.</summary>
/// <param name="company">The company to create.</param>
/// <response code="201">The company was created; the Location header points at it.</response>
/// <response code="401">No valid token was supplied.</response>
/// <response code="422">The payload parsed but failed validation.</response>
[HttpPost(Name = "CreateCompany")]
[Authorize(Roles = "Manager")]
[ProducesResponseType<CompanyDto>(StatusCodes.Status201Created)]
[ProducesResponseType<ProblemDetails>(StatusCodes.Status401Unauthorized)]
[ProducesResponseType<ValidationProblemDetails>(StatusCodes.Status422UnprocessableEntity)]
[ServiceFilter(typeof(ValidationFilterAttribute))]
public async Task<IActionResult> CreateCompany([FromBody] CompanyForCreationDto company)
{
    var created = await service.CompanyService.CreateCompanyAsync(company);
    return CreatedAtRoute("CompanyById", new { id = created.Id }, created);
}

```

Two habits keep this honest. Put `[Produces("application/json")]` and the error-shape attributes on a base controller so you write them once rather than thirty times. And treat the generated `swagger.json` as an artifact: check it into the build, diff it on every pull request, and a breaking change

stops being something a client discovers for you.

One caveat on exposure: the UI is a development and internal tool. Serve the document publicly if you want consumers to generate clients, but do not leave an interactive, credential-accepting UI on an internet-facing production API unless you have deliberately decided to.

Go deeper: the course's *Documenting APIs* chapter covers the Swashbuckle setup end to end, including the *Adding Authorization Support* lesson that wires the lock icon to your JWT.

What this chapter had to leave out

- The logging service built from an empty project: the `ILoggerManager` abstraction, Serilog configuration, and the Seq sink where structured logging finally shows what it is for.
- The complete health-check set: the basic probe, a SQL Server check, a custom `IHealthCheck`, tag filtering, and the health-checks UI dashboard.
- Swagger from first integration through to the authorization support that wires the lock icon to your JWT, so the docs page can actually call a protected endpoint.
- Extending the Swagger configuration past the defaults: multiple documents, versioned endpoints, and the XML comment plumbing that keeps them accurate.

Logging, health checks, and docs, built rather than described. [Get Ultimate ASP.NET Core Web API](#)

Chapter 7: Ship It

Everything up to this point makes the API correct. This chapter makes it survivable: a versioning story so the first breaking change is not a crisis, tests that exercise the real pipeline instead of a mock of it, configuration that differs per environment without leaking production values into your repository, and a deployment that gets its secrets from the host rather than from a file you committed. These four items are what separates an API that runs on your machine from one you can hand to other people.

7.1 Version your API from day one

Why it matters

The day you rename a field, change a response shape, or swap a status code, every client that depends on the old behaviour breaks. Without a versioning story you have exactly two options, and both are bad: break your consumers and spend a week apologising, or freeze the API forever and pile workarounds on top of it. Versioning costs you about fifteen minutes at the start of the project. Retrofitting it after you have live consumers costs you a release cycle.

Note that versioning is for *breaking* changes only. Adding an optional field or a new endpoint is not breaking, and does not need a new version. Renaming a property, removing one, changing a payload structure, changing which status code an operation returns, or redesigning a route: those are.

How to do it right

Install `Asp.Versioning.Mvc` (it replaces the deprecated `Microsoft.AspNetCore.Mvc.Versioning` package) and configure it in one extension method so `Program.cs` stays readable.

```
// ServiceExtensions.cs
using Asp.Versioning;

public static void ConfigureVersioning(this IServiceCollection services)
{
    services.AddApiVersioning(opt =>
    {
        opt.ReportApiVersions = true; // adds api-supported-versions to responses
        opt.AssumeDefaultVersionWhenUnspecified = true;
        opt.DefaultApiVersion = new ApiVersion(1, 0);
        opt.ApiVersionReader = new HeaderApiVersionReader("api-version");
    })
    .AddMvc();
}
```

```
// Program.cs
builder.Services.ConfigureVersioning();
```

You have three readers to choose from, and you pick one and stay with it:

Reader	Client sends	Trade-off
<code>HeaderApiVersionReader("api-version")</code>	a request header	URI stays stable; harder to test in a browser
<code>QueryStringApiVersionReader("api-version")</code>	<code>?api-version=2.0</code>	trivially testable; clutters every URL
URL segment, via <code>[Route("api/{v:apiversion}/companies")]</code>	<code>/api/2.0/companies</code>	most visible; two versions can never share a route

Mark the controllers themselves:

```
[ApiVersion("1.0")]
[Route("api/companies")]
[ApiController]
public class CompaniesController : ControllerBase { /* ... */ }

[ApiVersion("2.0")]
[Route("api/companies")]
[ApiController]
public class CompaniesV2Controller : ControllerBase { /* ... */ }
```

When a version reaches end of life, do not delete it out from under people. Deprecate it first, so `ReportApiVersions` starts advertising it as deprecated in the `api-deprecated-versions` response header, and clients get warned by the API itself before anything stops working:

```
[ApiVersion("2.0", Deprecated = true)]
```

Once you have more than a couple of versioned controllers, the attributes get noisy. Move the assignments into the configuration with conventions instead, and drop the attributes:

```
services.AddApiVersioning(opt => { /* as above */ })
    .AddMvc(opt =>
    {
        opt.Conventions.Controller<CompaniesController>()
            .HasApiVersion(new ApiVersion(1, 0));
        opt.Conventions.Controller<CompaniesV2Controller>()
            .HasDeprecatedApiVersion(new ApiVersion(2, 0));
    });
```

Two practical rules to hold yourself to. First, ship version 1.0 on day one even when there is only one version, because retrofitting the default onto a live API is where the breakage happens. Second, publish a sunset date with every deprecation, and make sure your OpenAPI document has a `SwaggerDoc` entry per version so consumers can actually read the difference.

Go deeper: *Versioning APIs* in the Ultimate ASP.NET Core Web API course walks through all three version readers with live requests, and covers deprecation and conventions.

7.2 Test the API through its real pipeline, not around it

Why it matters

Unit tests over a mocked repository tell you your mapping code compiles. They tell you nothing about routing, model binding, your validation filter, the `[Authorize]` attribute you forgot, the serializer settings, or the exception handler. Those are the parts that actually break in production. A suite that is 95% green while an endpoint returns 415 to every real client is not a safety net. It is a false one.

The other trap is testing against an in-memory database provider. It does not enforce your constraints, it does not behave like SQL Server on relationships or transactions, and it will happily let a test pass that fails the moment it hits a real database. If your test database is not the database you deploy to, your integration tests are a simulation of a system you do not run.

How to do it right

Use xUnit with `Microsoft.AspNetCore.Mvc.Testing` to boot the actual application, and Testcontainers to give every run a disposable, real SQL Server. You get production fidelity without a shared test database that everyone fights over.

```
dotnet add package xunit
dotnet add package xunit.runner.visualstudio
dotnet add package Microsoft.AspNetCore.Mvc.Testing --version 10.0.12
dotnet add package Testcontainers.MsSql
```

`WebApplicationFactory<T>` needs a reference to your `Program` class, which top-level statements make internal. Add this line at the end of `Program.cs`:

```
public partial class Program { }
```

Now build a factory that starts a container, points the `DbContext` at it, and migrates the schema:

```
public class ApiFactory : WebApplicationFactory<Program>, IAsyncLifetime
{
    private readonly MsSqlContainer _sql = new MsSqlBuilder()
        .WithImage("mcr.microsoft.com/mssql/server:2022-latest")
        .WithPortBinding(1433, assignRandomHostPort: true) // no port clashes in CI
        .WithWaitStrategy(Wait.ForUnixContainer().UntilPortIsAvailable(1433))
        .Build();

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {

```

```

        builder.ConfigureServices(services =>
        {
            services.RemoveAll(typeof(DbContextOptions<RepositoryContext>));

            services.AddDbContext<RepositoryContext>(options =>
                options.UseSqlServer(_sql.GetConnectionString())
                    .ConfigureWarnings(w =>
                        w.Ignore(RelationalEventId.PendingModelChangesWarning)));

            using var scope = services.BuildServiceProvider().CreateScope();
            scope.ServiceProvider.GetRequiredService<RepositoryContext>().Database.Migrate();
        });

        public async Task InitializeAsync() => await _sql.StartAsync();
        public new async Task DisposeAsync() => await _sql.DisposeAsync();
    }

```

Two details are not optional. `assignRandomHostPort: true` stops parallel runs and CI agents from colliding on port 1433. The wait strategy stops your first test from firing at a container that has not finished booting. Without it you get flaky failures that look like real bugs.

Then test through `HttpClient`, exactly as a client would. `IClassFixture<T>` reuses one container per test class instead of one per test:

```

public class CompaniesTests : IClassFixture<ApiFactory>
{
    private readonly HttpClient _client;
    private const string Url = "/api/companies";

    public CompaniesTests(ApiFactory factory) => _client = factory.CreateClient();

    [Fact]
    public async Task WhenNameIsMissing_ThenReturns422()
    {
        var company = new CompanyForCreationDto { Address = "Main St 1", Country = "USA" };

        var response = await _client.PostAsJsonAsync(Url, company);

        Assert.Equal(HttpStatusCode.UnprocessableEntity, response.StatusCode);
    }

    [Fact]
    public async Task WhenNotAuthenticated_ThenReturns401()
    {
        var response = await _client.GetAsync(Url);

        Assert.Equal(HttpStatusCode.Unauthorized, response.StatusCode);
    }
}

```

Those two tests are worth more than a dozen mocked service tests, because they cover the validation filter and the authorization attribute: the wiring, not the logic. Cover the boring status codes deliberately: 401, 404, 415, 422, and a 201 with a non-null `Location` header.

Testcontainers needs Docker installed on whatever machine runs the tests, including your CI agent. That is the whole dependency. You do not containerise your application to use it.

Go deeper: *Testing ASP.NET Core Web API* in the course argues the in-memory vs. real vs. Testcontainers trade-off explicitly, then builds the factory and the validation, authorization, and paging test classes on top of it.

7.3 Separate configuration per environment and keep production values out of the repo

Why it matters

One production connection string committed to a public repository is a full compromise, and rotating it afterwards means an outage. The second failure mode is quieter: behaviour that differs between environments but is only visible in a stringly-typed lookup buried in a service. A

`Configuration["JwtSettings:Expires"]` typo does not fail the build, does not fail startup, and does not fail your tests. It fails at 3am when someone tries to log in, having silently returned `null` for a month.

How to do it right

ASP.NET Core layers configuration for you. `appsettings.json` holds the shared, non-secret defaults. `appsettings.{Environment}.json` overrides individual keys for that environment, selected by the `ASPNETCORE_ENVIRONMENT` variable. Anything genuinely secret comes from outside the file system entirely.

<code>appsettings.json</code>	→ shared defaults, no secrets, committed
<code>appsettings.Development.json</code>	→ local ports, verbose logging, committed
<code>appsettings.Production.json</code>	→ structure only, no real values, committed
user secrets / host env vars	→ the actual secrets, never committed

The production file must exist even when it is empty, because a missing one is a startup failure on the host. `{}` is a valid file.

In development, use user secrets rather than editing `appsettings.Development.json`, because that file is in your repository:

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:SqlConnection"
"Server=.;Database=CompanyEmployee;..."
```

In production, the host supplies the same keys as environment variables, using `__` for nesting (`ConnectionStrings__SqlConnection`). The configuration system merges them over the JSON files with no code change on your side.

Then bind the settings to a class instead of reaching for string keys, so a typo is a startup problem and not a runtime one:

```
public class JwtConfiguration
{
    [Required] public string ValidIssuer { get; set; } = default!;
    [Required] public string ValidAudience { get; set; } = default!;
    [Range(1, 60)] public int Expires { get; set; }
}
```

```
builder.Services.AddOptions<JwtConfiguration>()
    .Bind(builder.Configuration.GetSection("JwtSettings"))
    .ValidateDataAnnotations()
    .ValidateOnStart();           // fail fast, at boot, with a readable message
```

`ValidateOnStart()` is the whole point: a missing or malformed setting crashes the process on deploy, when you are watching, rather than on the first request that needs it.

Consume it through `IOptions<T>` in most places. Reach for the other two only when you have a reason:

- `IOptions<T>`: singleton, bound once at startup, never reloads. The default.
- `IOptionsSnapshot<T>`: scoped, re-reads per request. Use for values you want to change without a restart. Cannot be injected into a singleton.
- `IOptionsMonitor<T>`: singleton, reloads immediately, exposes `CurrentValue` instead of `Value`. Use when a singleton needs live values.

Finally, keep environment branching thin and honest. This is fine:

```
if (app.Environment.IsDevelopment())
    app.UseDeveloperExceptionPage();
else
    app.UseHsts();
```

Anything more than that (different business rules, different validation, whole features toggled by environment) means you are shipping code paths to production that no environment ever tested.

Go deeper: *Environment-Based Settings* in the Project Configuration section covers the file layering and `ASPNETCORE_ENVIRONMENT`; *Binding Configuration and Options Pattern* covers typed options and the `IOptionsSnapshot` / `IOptionsMonitor` distinction.

7.4 Deploy from a published build and let the host supply the secrets

Why it matters

"It runs when I press F5" is not a deployment. A debug build carries symbols, resource satellites, and development configuration you do not want on a server, and it is running under a launch profile that will not exist on the host. Meanwhile, every configuration value that has only ever come from your local machine (a signing key, a connection string) becomes a startup crash the first time you publish. That crash is usually invisible too: IIS returns a bare 500.30 and your logs are empty, because the process died before logging was configured.

Deploy early and deploy often, even if it is only to a local IIS site. The first deployment should happen in week one, not the week before launch.

How to do it right

Publish a Release build to a folder. The wizard in Visual Studio does this, but do it from the CLI so it is scriptable and identical on your CI server:

```
dotnet publish ./CompanyEmployees/CompanyEmployees.csproj -c Release -o ./Publish
```

Trim what you do not want shipped, in the main project's `.csproj` (repeat the debug properties in each project to fully suppress `.pdb` files):

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <SatelliteResourceLanguages>en</SatelliteResourceLanguages>
  <DebugType>None</DebugType>
  <DebugSymbols>>false</DebugSymbols>
  <ErrorOnDuplicatePublishOutputFiles>>false</ErrorOnDuplicatePublishOutputFiles>
</PropertyGroup>
```

Install the ASP.NET Core Hosting Bundle on the host. IIS cannot run your application on its own. The bundle installs the .NET runtime plus `AspNetCoreModuleV2`, which is what actually makes IIS a reverse proxy in front of Kestrel. Without it you get a 500.19 or a 502.5 and no useful message.

The publish output includes a generated `web.config`. It is worth understanding, because it is where first-boot diagnosis happens:

```
<aspNetCore processPath="dotnet"
  arguments=".\\CompanyEmployees.dll"
  hostingModel="inprocess"
  stdoutLogEnabled="false"
  stdoutLogFile=".\logs\stdout" />
```

`hostingModel="inprocess"` runs your app inside the IIS worker process, which is faster, and the default. `outofprocess` starts a separate Kestrel process that IIS proxies to. Flip `stdoutLogEnabled` to `true` temporarily when the app will not start: it captures the startup exception that IIS swallows. Turn it back off once you are done, because it grows without bound.

Let the host supply the secrets. Nothing sensitive travels in the publish folder. In IIS, open your site, then Configuration Editor, choose the `system.webServer/aspNetCore` section, switch the *From* dropdown to `ApplicationHost.config`, open the `environmentVariables` collection, and add the entries:

```
SECRET = <the JWT signing key>
ASPNETCORE_ENVIRONMENT = Production
ConnectionStrings__SqlConnection = <the production connection string>
```

Click Apply and restart the site. The application reads them exactly like any other configuration source, with no code change between environments.

Make sure the app pool identity can reach the database. If you use Windows authentication on the connection string, add a SQL Server login for `IIS APPPOOL\<YourAppPoolName>` and map it to the database with read and write roles. This is the single most common reason a deployment that starts fine returns 500 on the first real request.

Read the proxy headers. Behind IIS, or any reverse proxy, every request reaches Kestrel over plain HTTP from localhost, so `Request.Scheme` says `http` and the client IP is the proxy's. Register the forwarded-headers middleware first, ahead of anything that inspects scheme, host, or IP:

```
var app = builder.Build();

app.UseForwardedHeaders(new ForwardedHeadersOptions
{
    ForwardedHeaders = ForwardedHeaders.All
});

if (!app.Environment.IsDevelopment())
    app.UseHsts();

app.UseHttpsRedirection();
```

Put `UseHttpsRedirection` before the forwarded headers are read and it will redirect requests that already arrived over HTTPS, a redirect loop that only reproduces on the server.

Verify, do not assume. A deployment is done when you have hit the deployed URL, authenticated against it, called a protected endpoint with the resulting token, and seen `/health` report `Healthy` (item 6.2). Anything short of that is a deployment you hope worked.

Go deeper: *Deployment to IIS* in the course walks the full flow: publish folder, Hosting Bundle and IIS site setup, the environment variables in `ApplicationHost.config`, and testing the deployed application end to end.

What this chapter had to leave out

- All three version readers demonstrated against live requests, plus deprecating a version and the conventions that keep the routing readable once you have three of them.
- The full test suite: xUnit set up with Testcontainers, then validation, authorization, and paging/filtering/searching/sorting test classes built on top of it.
- Deployment end to end: the publish folder, the Windows Server Hosting Bundle, the IIS site, the environment variables in `ApplicationHost.config`, and testing the deployed application against a real token.
- The bonus modules you reach for after the first release: CQRS with MediatR, and working with multiple databases behind the same repository layer.
- The appendix of HTTP reference tables (request methods, status codes, request headers, and MIME types) in one place you can keep open while you work.
- **Dockerizing ASP.NET Core Applications**, a second course, free with your purchase, for the containerised path to production.

42 topics, 194 lessons, all the source code, and two bonus courses. [Get Ultimate ASP.NET Core Web API](#)

Where to go next

What you have now, and what production will still throw at you

You have 33 items and the code to implement each one. That is a real thing to have. Run this list against an API before you ship it and you will not make the mistakes that put most first production APIs on a postmortem page: the unbounded list endpoint, the singleton holding a scoped `DbContext`, the wide-open CORS policy, the 500 that should have been a 422, the signing secret sitting in `appsettings.json`.

A checklist has a limit. It tells you what the finished decision looks like. It cannot tell you what to do when reality does not match the sample: when the refresh-token flow has to work across a mobile client and a SPA at the same time, when the paging query is correct and still slow, when Identity is wired in but the login endpoint returns 401 for a user you can see in the database, when the deployed app starts fine and returns 500 on the first real request. Those are the days you learn this, and they go a great deal faster with someone showing you the way through.

That is the gap between this book and the course. Not more items. The reasoning, the failures, and the twenty minutes of refactoring that arrived at each answer.

The complete path: Ultimate ASP.NET Core Web API

Every "Go deeper" pointer in this book goes to one place: [Ultimate ASP.NET Core Web API](#), with **42 topics, 194 lessons, and the full source code for every one of them.**

The course builds one real application, companies and their employees, the same example running through these pages, from an empty solution to a deployed, tested, secured, documented API, and you write it alongside the lessons. Every chapter you just read points into it. Every box marked *What this chapter had to leave out* is in there in full:

- **The whole onion build**, project by project, including what breaks at each step and why the presentation assembly needs `AddApplicationPart`.
- **Nine lessons on validation**, four on action filters, and the PUT and PATCH paths handled as the different problems they are.
- **The complete authentication and refresh-token story**: Identity tables, role seeding, user creation, protected endpoints, role-based authorization, token rotation, and the options pattern behind the JWT configuration.
- **Paging, filtering, searching, sorting and caching** built one module at a time, with the generated SQL on screen and the count-before-skip optimisation timed against millions of rows.
- **Testing in depth**: Testcontainers with xUnit, then validation, authorization, and paging/filtering/searching/sorting test classes on top of it.

- **The topics this book deliberately left out:** HATEOAS, data shaping, content negotiation and custom formatters, a root document, custom vendor media types, CQRS with MediatR, discriminated unions as an exception-free result flow, multiple databases, and an appendix of HTTP reference tables.
- **Every lesson's source code**, so you can diff your version against the finished one the moment something does not line up.

If you worked through this book and found yourself wanting the reasoning behind a decision rather than just the decision, that is exactly the course's job.

Two more courses, free with it

Buying Ultimate ASP.NET Core Web API also gets you two complete courses at no extra cost. Both are full courses, not samples or trailers, and they cover the two areas this book could only gesture at.

Mastering ASP.NET Core Security is free with your purchase. Chapter 4 gave you four working JWT validation parameters, a secret kept out of source control, Identity, authorization, refresh tokens and CORS. This course is where security stops being a chapter and becomes the subject: everything that comes after the token is issued.

Dockerizing ASP.NET Core Applications is free with your purchase. You will have noticed there is no Docker item in this checklist, and that was deliberate: containerisation is too large for a single entry, and it is already covered properly here. Images, layers, Dockerfiles that do not ship your secrets, Compose for local dependencies, and the parts of the workflow that only make sense once you have deployed something without containers first. Having read item 7.4, you now have. It also closes a loop from item 7.2: Testcontainers needs Docker on the machine running your tests, and knowing what a container actually is makes that dependency much less mysterious.

Get the course

One purchase. Three courses. The complete path from an empty solution to an API you can hand to other people.

Get Ultimate ASP.NET Core Web API

About Code Maze

Code Maze is a .NET blog and course platform. We write about ASP.NET Core, C#, Entity Framework Core, testing, and architecture, in the same style as this book: concrete problems, working code, no filler. Everything on the blog is free and there is no sign-up wall on it.

You can find it at code-maze.com, and the courses at courses.code-maze.com.

Thanks for reading. Go ship something.